



GRADO EN INGENIERÍA
INFORMÁTICA



VNIVERSITAT
DE VALÈNCIA

TRABAJO FIN DE GRADO

DESARROLLO DE UNA TIENDA ONLINE PARA UNA
DROGUERÍA

AUTOR: CARLOS JAVIER PIQUER MAÑEZ

TUTOR: ANDRES ROCAFULL RODRIGO

MARZO 2026



VNIVERSITAT
DE VALÈNCIA



Escola Tècnica Superior
d'Enginyeria **ETSE-UV**

TRABAJO FIN DE GRADO

DESARROLLO DE UNA TIENDA ONLINE PARA UNA DROGUERÍA

AUTOR: CARLOS JAVIER PIQUER MAÑEZ

TUTOR: ANDRES ROCAFULL RODRIGO

Declaración de autoría:

Yo, Carlos Javier Piquer Mañez, declaro la autoría del Trabajo Fin de Grado titulado “Desarrollo de una tienda online para una droguería” y que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual. El material no original que figura en este trabajo ha sido atribuido a sus legítimos autores.

Valencia, 4 de junio de 2026

Fdo: Carlos Javier Piquer Mañez

Resumen:

Este es el resumen del TFM. Debe ser corto (máximo media página) y cubrir los aspectos principales del TFM.

Abstract:

This is the abstract of the TFM. It must be short and cover the main aspects of the TFM.

Resum:

Aquest és el resum del TFM. Ha de ser curt (màxim mitja pàgina) i cobrir els aspectes principals del TFM.

Agradecimientos:

En primer lugar quiero agradecer a todos aquellos que me han apoyado durante todos estos años.

En segundo lugar...

Índice general

1. Introducción	19
1.1. Introducción	19
1.2. Motivación	19
1.3. Objetivos	20
1.4. Organización de la memoria	20
2. Estado del arte	23
2.1. Análisis de aplicaciones similares	23
2.1.1. Wix eCommerce	23
2.1.2. Squarespace	24
2.1.3. PrestaShop	24
2.1.4. Comparación con la propuesta del TFG	25
2.2. Tecnologías	25
2.2.1. Frameworks frontend	25
2.2.2. Frameworks backend	27
2.2.3. Bots de Telegram	29
2.2.4. Bases de datos	31
2.2.5. Pasarelas de pago	32
3. Requisitos, especificaciones, coste, riesgos, viabilidad	35
3.1. Requisitos	35
3.1.1. Requisitos funcionales	35
3.1.2. Requisitos no funcionales	36
3.2. Especificaciones	36
3.3. Tareas y costes	37
3.3.1. Definición de tareas	37
3.3.2. Estimación temporal	38
3.3.3. Diagrama de Gantt	39
3.3.4. Estimación de costes	41

3.4.	Riesgos	43
3.5.	Viabilidad	44
3.5.1.	Viabilidad legal	44
3.5.2.	Viabilidad técnica	44
3.5.3.	Viabilidad económica	44
4.	Análisis	45
4.1.	Diagramas de análisis	45
4.1.1.	Diagrama de casos de uso	45
4.1.2.	Diagrama de clases de análisis	46
4.1.3.	Diagrama de secuencia general del sistema (DSGS)	47
5.	Diseño	49
5.1.	Arquitectura general del sistema	49
5.2.	Diagramas UML	49
5.2.1.	Diagrama de clases	49
5.2.2.	Diagramas de secuencia	51
5.3.	Diseño de la base de datos	58
5.3.1.	Modelo entidad-relación	58
5.3.2.	Modelo relacional y prototipo inicial	59
5.4.	Diseño de la interfaz de usuario	60
5.4.1.	Prototipo de la página de inicio	60
5.4.2.	Prototipo del catálogo de productos	61
5.4.3.	Prototipo del carrito de compra	62
5.4.4.	Prototipo del proceso de pago (<i>Checkout</i>)	63
6.	Implementación y pruebas	65
6.1.	Implementación	65
6.1.1.	Creación de pedido	65
6.1.2.	Recuperación de contraseña	66
6.1.3.	Estadísticas del panel de administración	67
6.1.4.	Gestión de imágenes con Cloudinary	67
6.2.	Pruebas funcionales	69
6.2.1.	Registro y autenticación (RF-01)	69
6.2.2.	Catálogo y carrito (RF-02, RF-03)	69
6.2.3.	Proceso de compra (RF-04, RF-06)	70
6.2.4.	Administrador (RF-05, RF-08)	70
6.2.5.	Bot de Telegram (RF-07, RF-09)	70

6.3. Pruebas de rendimiento	71
6.3.1. Rendimiento del frontend — Google Lighthouse	71
6.3.2. Rendimiento del backend — Tiempos de respuesta de la API	72
6.4. Pruebas de usabilidad	72
6.4.1. Participantes y tareas	72
6.4.2. Resultados	73
7. Conclusiones	75
7.1. Revisión de costes	75
7.2. Conclusiones	75
7.3. Trabajo futuro	76
Bibliografía	77

Capítulo 1

Introducción

1.1. Introducción

El presente trabajo de fin de grado describe el desarrollo de una **tienda online** para la **Droguería Mercería Inés**, un negocio familiar ubicado en Valencia. El establecimiento, regentado por Ester, ofrece una amplia variedad de productos: perfumes, utensilios de costura, hilos, telas especiales y artículos de mercería en general, siguiendo la tradición de las droguerías de barrio que durante décadas han formado parte del comercio local.

Aunque el negocio cuenta con presencia en redes sociales, no disponía de una página web propia que permitiese a sus clientes consultar el catálogo, realizar pedidos y efectuar pagos de forma online. Este proyecto nace con el objetivo de cubrir esa necesidad mediante el desarrollo de una aplicación web completa, construida íntegramente con herramientas gratuitas y de código abierto.

La solución desarrollada permite a los clientes registrarse, explorar el catálogo de productos, gestionar su carrito y completar compras a través de la pasarela de pago *Stripe*. Por parte de la administración, el sistema facilita la gestión del inventario y el seguimiento de pedidos, con notificaciones automáticas de confirmación de compra y avisos de bajo stock a través de un *bot de Telegram*.

1.2. Motivación

La principal motivación de este proyecto es de carácter personal: dar respuesta a una necesidad real de un negocio familiar. Desarrollar esta solución supone una aportación directa al negocio de la familia, facilitando tanto la experiencia de compra de los clientes como el trabajo diario de la propietaria en la gestión del stock y los pedidos.

Además, existe una motivación de carácter profesional. La elección de **Angular** y **Spring Boot** como tecnologías principales responde al hecho de que son las herramientas utilizadas en **Capgemini**, empresa en la que se van a realizar las prácticas. Abordar este proyecto ha permitido adquirir una base sólida en ambas tecnologías antes de incorporarse a un entorno profesional, complementando la formación académica con una experiencia de desarrollo real y completa.

Por último, este proyecto pretende demostrar que es posible construir una solución de comercio electrónico funcional y profesional haciendo uso exclusivamente de herramientas

gratuitas y de código abierto, sin necesidad de recurrir a plataformas de pago ni a infraestructuras costosas.

1.3. Objetivos

El objetivo principal del proyecto es el siguiente:

- **Desarrollar una tienda online funcional y completa para la Droguería Mercería Inés**, que permita a sus clientes consultar el catálogo, realizar pedidos y efectuar pagos de forma segura a través de internet.

Para alcanzarlo, se han definido los siguientes objetivos secundarios:

- Dar visibilidad online al negocio mediante una plataforma web propia, complementando su presencia actual en redes sociales.
- Facilitar a la propietaria el control del inventario y el seguimiento de los pedidos recibidos a través de la aplicación.
- Automatizar las notificaciones de confirmación de pedido y los avisos de bajo stock mediante un bot de Telegram integrado en el sistema.
- Adquirir experiencia práctica en **Angular** y **Spring Boot** como preparación para las prácticas profesionales en Capgemini.
- Demostrar que una solución de comercio electrónico completa puede desarrollarse íntegramente con herramientas gratuitas y de código abierto.

1.4. Organización de la memoria

La memoria se organiza en los siguientes capítulos:

- **Capítulo 1 — Introducción:** presenta el proyecto, su motivación, los objetivos planteados y la estructura del documento.
- **Capítulo 2 — Estado del arte:** analiza las plataformas de creación de tiendas online más representativas y estudia las tecnologías seleccionadas para el desarrollo del sistema, justificando cada elección.
- **Capítulo 3 — Requisitos, especificaciones, coste, riesgos y viabilidad:** recoge los requisitos funcionales y no funcionales del sistema, las especificaciones técnicas, la planificación temporal y económica del proyecto, los riesgos identificados y el análisis de viabilidad.
- **Capítulo 4 — Análisis:** incluye los diagramas de análisis del sistema: el diagrama de casos de uso, el diagrama de clases de análisis y los diagramas de secuencia general del sistema (DSGS).

- **Capítulo 5 — Diseño:** describe la arquitectura general del sistema, los diagramas UML de diseño, el modelo de la base de datos y los prototipos de la interfaz de usuario.
- **Capítulo 6 — Implementación y pruebas:** detalla el proceso de desarrollo de la aplicación y los resultados de las pruebas funcionales, de rendimiento y de usabilidad realizadas.
- **Capítulo 7 — Conclusiones:** revisa los costes finales del proyecto, presenta las conclusiones obtenidas y propone posibles líneas de trabajo futuro.

Capítulo 2

Estado del arte

2.1. Análisis de aplicaciones similares

En el mercado actual la gran variedad de plataformas que nos ofrecen servicios para crear nuestra propia tienda online sin necesidad de tener algún tipo de conocimiento sobre programar es muy extenso. Algunas de las más representativas son:

2.1.1. Wix eCommerce

Wix proporciona un sistema de creación de páginas basado en el uso de herramientas de arrastrar y soltar, lo que facilita a cualquier usuario construir una tienda en poco tiempo [1]. Sin embargo, esta simplicidad también conlleva limitaciones, especialmente en lo que respecta a la personalización avanzada y al control sobre el backend, aspectos que resultan esenciales en proyectos con requisitos específicos.

Ventajas:

- Interfaz intuitiva y sistema de diseño drag-and-drop.
- Plantillas atractivas y actualizadas.
- Incluye hosting y mantenimiento automático.
- Integración sencilla con herramientas externas.

Desventajas:

- Escasa flexibilidad para personalizaciones complejas.
- Limitaciones en el acceso al código fuente.
- Dificultades para escalar proyectos grandes.
- Dependencia total del ecosistema de Wix.

2.1.2. Squarespace

Squarespace está orientado a usuarios que priorizan el diseño visual y la facilidad de uso [2]. Sus plantillas resultan intuitivas y permiten crear páginas atractivas sin dificultad. No obstante, su ecosistema de integraciones es más limitado, lo que restringe su aplicabilidad en proyectos que requieren funcionalidades técnicas más avanzadas.

Ventajas:

- Excelente diseño visual y acabados profesionales.
- Plataforma todo en uno (hosting, dominio y soporte).
- Adaptación automática a dispositivos móviles.
- Seguridad gestionada por la propia plataforma.

Desventajas:

- Pocas integraciones con APIs o sistemas externos.
- Costes mensuales relativamente altos.
- Escasa libertad para modificar la estructura del sitio.
- Dependencia del entorno cerrado de Squarespace.

2.1.3. PrestaShop

PrestaShop es una solución de código abierto muy popular para la creación de tiendas online [3]. Ofrece una gran capacidad de personalización mediante módulos y plantillas, lo que la convierte en una opción atractiva para pequeñas y medianas empresas. No obstante, requiere ciertos conocimientos técnicos para su instalación y mantenimiento, y en proyectos de mayor tamaño puede presentar dificultades de rendimiento si no se configura correctamente.

Ventajas:

- Open source y altamente personalizable.
- Gran comunidad de desarrolladores.
- Amplio catálogo de módulos y temas.
- Permite control total sobre el servidor y la base de datos.

Desventajas:

- Necesita conocimientos técnicos para instalación y soporte.
- Requiere mantenimiento y actualizaciones manuales.
- Puede presentar problemas de rendimiento en tiendas grandes.
- Algunos módulos son de pago y elevan los costes.

2.1.4. Comparación con la propuesta del TFG

Las plataformas analizadas muestran distintas formas de crear una tienda online sin necesidad de conocimientos de programación. Wix y Squarespace destacan por su sencillez y diseño intuitivo, aunque sacrifican flexibilidad y control sobre la aplicación. Por su parte, PrestaShop ofrece mayor capacidad de personalización, pero exige un nivel técnico más elevado y puede presentar limitaciones de rendimiento en proyectos de gran escala. En contraste, la propuesta de este TFG plantea el desarrollo de una solución a medida que permita un mayor control sobre la arquitectura, la seguridad y las integraciones, adaptándose específicamente a las necesidades del proyecto.

Criterio	Wix	Squarespace	PrestaShop
Facilidad de uso	Muy alta	Alta	Media
Personalización	Baja	Media	Alta
Escalabilidad	Limitada	Media	Alta
Requiere conocimientos técnicos	No	No	Sí
Coste	Medio	Medio	Variable

Cuadro 2.1: Comparativa de plataformas de creación de tiendas online

2.2. Tecnologías

2.2.1. Frameworks frontend

Angular

Angular es un framework desarrollado por Google y basado en TypeScript [4]. Ofrece una solución completa para el desarrollo de aplicaciones web a gran escala, con herramientas integradas como enrutamiento y gestión de formularios. Su principal desventaja es la necesidad de contar con conocimientos previos más avanzados, lo que incrementa la curva de aprendizaje.

Ventajas:

- Framework completo con soporte oficial de Google.
- Tipado fuerte con TypeScript, que mejora la robustez del código.
- Soporte integrado para testing, formularios y enrutamiento.
- Comunidad amplia y documentación exhaustiva.

Desventajas:

- Curva de aprendizaje pronunciada.
- Tamaño inicial del proyecto elevado.
- Requiere experiencia previa en TypeScript.
- Sintaxis más compleja frente a otras alternativas.

React

React es una tecnología desarrollada por Meta [5]. Se trata de una biblioteca para la construcción de interfaces de usuario, que destaca por su flexibilidad y por el rendimiento que ofrece gracias al uso del Virtual DOM. Sin embargo, al no ser un framework completo, depende de librerías de la comunidad para cubrir funcionalidades adicionales o más avanzadas.

Ventajas:

- Excelente rendimiento gracias al Virtual DOM.
- Gran comunidad y recursos de aprendizaje.
- Compatible con múltiples librerías y frameworks.
- Curva de aprendizaje accesible para principiantes.

Desventajas:

- Necesidad de integrar herramientas externas (routing, estado).
- Menor estructura definida que Angular.
- Frecuentes actualizaciones y cambios en librerías.
- Requiere experiencia para mantener proyectos grandes.

Vue

Vue es un framework progresivo creado por Evan You, exingeniero de Google [6]. Combina características de Angular y React, con un enfoque en la simplicidad y en la rapidez de aprendizaje. No obstante, su comunidad y ecosistema siguen siendo más reducidos en comparación con otras tecnologías consolidadas.

Ventajas:

- Ligero y de fácil adopción.
- Documentación clara y organizada.
- Buena integración con proyectos existentes.
- Sintaxis sencilla y curva de aprendizaje rápida.

Desventajas:

- Ecosistema más limitado que Angular o React.
- Menor presencia en grandes empresas.
- Menos recursos de formación avanzada.
- Riesgo de menor soporte a largo plazo.

Comparación y elección

Los tres frameworks permiten crear aplicaciones modernas, pero Angular ofrece una estructura más completa desde el inicio. Para este TFG se ha optado por Angular, ya que es la tecnología utilizada en Capgemini, empresa en la que se van a realizar las prácticas. De esta forma, el proyecto servirá también como una preparación práctica, complementada con cursos de formación, para no comenzar esa experiencia sin conocimientos previos.

Criterio	Angular	React	Vue
Lenguaje principal	TypeScript	JavaScript	JavaScript
Complejidad inicial	Alta	Media	Baja
Ecosistema	Muy amplio	Amplio	Moderado
Soporte empresarial	Alto	Alto	Medio
Rendimiento general	Alto	Muy alto	Alto

Cuadro 2.2: Comparativa de frameworks frontend

2.2.2. Frameworks backend

Spring Boot

Spring Boot es un framework del ecosistema Java que facilita la creación de aplicaciones backend robustas y escalables [7]. Entre sus principales ventajas se encuentran su amplia comunidad, la integración con múltiples librerías y la facilidad para configurar proyectos complejos. Como inconvenientes, requiere conocimientos previos en Java y, según el tamaño de la aplicación, puede resultar más pesado que otras alternativas.

Ventajas:

- Framework maduro con gran soporte empresarial.
- Integración nativa con bases de datos y APIs REST.
- Facilita pruebas y despliegues automáticos.
- Gran estabilidad y seguridad.

Desventajas:

- Requiere conocimientos avanzados en Java.
- Consumo de recursos más alto que Node.js.
- Curva de aprendizaje larga.
- Configuraciones iniciales complejas.

Node.js

Node.js es un entorno de ejecución de JavaScript que permite utilizar este lenguaje en la configuración y desarrollo de servidores [8]. Destaca por su rendimiento en aplicaciones

en tiempo real y por la ventaja de compartir el mismo lenguaje en frontend y backend, lo que agiliza el desarrollo. Su principal limitación es que no está tan orientado a aplicaciones de gran escala como otros frameworks más consolidados.

Ventajas:

- Excelente rendimiento en tiempo real.
- Usa el mismo lenguaje en cliente y servidor.
- Amplia comunidad y soporte.
- Ecosistema extenso con NPM.

Desventajas:

- Menor rendimiento en tareas muy pesadas.
- Dificultad para proyectos altamente estructurados.
- Depende mucho de librerías externas.
- Escalabilidad más limitada que Spring Boot.

Django

Django es un framework de Python que sigue el principio "batteries included", lo que significa que incorpora numerosas funcionalidades listas para usar [9]. Es adecuado para el desarrollo rápido de prototipos y proyectos de tamaño medio, con una curva de aprendizaje moderada. Sin embargo, su rendimiento puede verse limitado en aplicaciones muy exigentes.

Ventajas:

- Gran cantidad de herramientas integradas.
- Excelente para desarrollo rápido.
- Seguridad y control de autenticación robusto.
- Documentación muy completa.

Desventajas:

- Menor rendimiento en aplicaciones grandes.
- Menor integración con sistemas Java.
- No tan extendido en entornos empresariales.
- Difícil de optimizar para proyectos complejos.

Comparación y elección

Spring Boot, Node.js y Django son alternativas viables para el desarrollo de un backend. Node.js destaca por su rapidez en tiempo real y Django por la simplicidad de Python, pero para este TFG se ha optado por Spring Boot. Esta elección responde a que es la tecnología utilizada en Capgemini, empresa en la que se realizarán las prácticas, lo que permite formarse previamente mediante cursos y adquirir los conceptos básicos necesarios. Además, Spring Boot destaca por su robustez, su madurez en entornos empresariales y porque complementa de forma óptima la arquitectura planteada con Angular y SQL.

Criterio	Spring Boot	Node.js	Django
Lenguaje base	Java	JavaScript	Python
Escalabilidad	Alta	Media	Media
Curva de aprendizaje	Alta	Media	Baja
Rendimiento general	Alto	Alto	Medio
Soporte empresarial	Muy alto	Medio	Medio

Cuadro 2.3: Comparativa de frameworks backend

2.2.3. Bots de Telegram

Los bots de Telegram son programas automatizados que interactúan con usuarios a través de la plataforma de mensajería Telegram mediante su API oficial. Permiten enviar notificaciones, gestionar comandos y ejecutar acciones en respuesta a eventos externos, lo que los convierte en una alternativa práctica para integrar comunicaciones automatizadas en aplicaciones web. A continuación se analizan las principales librerías disponibles para su desarrollo.

python-telegram-bot

python-telegram-bot es una librería de código abierto escrita en Python que proporciona una interfaz completa para interactuar con la API oficial de Telegram [10]. Ofrece soporte tanto para *polling* como para *webhooks*, y su diseño basado en *handlers* facilita la estructuración del código. Sin embargo, al estar escrita en Python, su integración con proyectos Java como Spring Boot requiere desplegar un servicio adicional.

Ventajas:

- Librería madura y bien documentada.
- Soporte completo para la API de Telegram.
- Gran comunidad activa y amplia cantidad de ejemplos.
- Compatible con *polling* y *webhooks*.

Desventajas:

- Requiere un servicio Python independiente si el backend es Java.
- Mayor complejidad de despliegue en proyectos políglotas.

- Gestión de dependencias separada del proyecto principal.
- Puede presentar latencias si no se configura correctamente el *webhook*.

Telegraf

Telegraf es un framework moderno para el desarrollo de bots de Telegram basado en Node.js y TypeScript [11]. Destaca por su arquitectura orientada a *middleware* y su sintaxis clara y expresiva, lo que facilita la creación de bots complejos con pocas líneas de código. Al igual que python-telegram-bot, requiere un entorno de ejecución independiente cuando se combina con un backend Java.

Ventajas:

- Sintaxis moderna y expresiva basada en *middleware*.
- Soporte nativo para TypeScript.
- Amplio ecosistema de plugins y extensiones.
- Buena documentación y comunidad activa.

Desventajas:

- Requiere entorno Node.js separado del backend Java.
- Curva de aprendizaje mayor si no se tiene experiencia con Node.js.
- Dependencia del ecosistema NPM.
- Configuración adicional para despliegues en producción.

TelegramBots (Java)

TelegramBots es una librería Java de código abierto que implementa la API de Telegram directamente en el ecosistema Java [12]. Su integración nativa con Spring Boot permite incorporar el bot dentro del mismo proyecto sin necesidad de servicios adicionales, simplificando el despliegue y el mantenimiento. Es la opción más coherente para proyectos backend basados en Java.

Ventajas:

- Integración directa con Spring Boot sin servicios adicionales.
- Mismo lenguaje y entorno que el resto del backend.
- Soporte completo para la API de Telegram.
- Simplifica el despliegue al unificar bot y backend en un solo proyecto.

Desventajas:

- Comunidad más reducida que las alternativas en Python o Node.js.

- Documentación menos extensa en comparación con `python-telegram-bot`.
- Menor cantidad de ejemplos y recursos de aprendizaje.
- Actualizaciones menos frecuentes que otras librerías.

Comparación y elección

Las tres opciones analizadas permiten desarrollar bots funcionales para Telegram, pero difieren en el lenguaje y el entorno de ejecución requerido. `python-telegram-bot` y `Telegraf` destacan por su popularidad y documentación, aunque ambas necesitan un servicio independiente cuando el backend está desarrollado en Java. Para este TFG se ha optado por **TelegramBots (Java)** por su integración nativa con Spring Boot, lo que permite incorporar el bot directamente en el mismo proyecto sin añadir complejidad de despliegue. Esta elección mantiene la coherencia tecnológica de la arquitectura y facilita el mantenimiento del sistema.

Criterio	<code>python-telegram-bot</code>	<code>Telegraf</code>	TelegramBots (Java)
Lenguaje	Python	Node.js / TypeScript	Java
Integración con Spring Boot	Indirecta	Indirecta	Nativa
Facilidad de uso	Alta	Alta	Media
Documentación	Muy completa	Completa	Limitada
Despliegue en proyecto Java	Complejo	Complejo	Simple

Cuadro 2.4: Comparativa de librerías para bots de Telegram

2.2.4. Bases de datos

SQL

Las bases de datos relacionales, como MySQL o PostgreSQL, utilizan un modelo basado en tablas y relaciones entre ellas [13]. Destacan por su consistencia, su alto nivel de seguridad y por el uso del lenguaje SQL, que es un estándar ampliamente adoptado en el entorno empresarial.

NoSQL

Las bases de datos NoSQL, como MongoDB, almacenan la información en estructuras más flexibles, como documentos o grafos. Ofrecen mayor escalabilidad y rapidez en ciertas operaciones, aunque sacrifican consistencia y mecanismos de seguridad más avanzados. [14].

Comparación y elección

Para este TFG se ha optado por utilizar **MySQL** como sistema de gestión de bases de datos. La elección se debe a su estabilidad, su integración nativa con Spring Boot y su capacidad para garantizar la seguridad de los datos. Además, a lo largo de la carrera se ha trabajado ampliamente con bases de datos SQL, mientras que NoSQL apenas se ha

tratado, por lo que MySQL resulta una opción más familiar y adecuada para el desarrollo del proyecto.

Criterio	SQL (MySQL)	NoSQL (MongoDB)
Estructura	Tablas relacionales	Documentos / grafos
Consistencia	Alta	Media
Seguridad	Alta	Media
Escalabilidad	Media	Alta
Soporte empresarial	Muy alto	Medio

Cuadro 2.5: Comparativa entre bases de datos SQL y NoSQL

2.2.5. Pasarelas de pago

PayPal

PayPal es una de las plataformas de pago online más conocidas y utilizadas a nivel mundial [15]. Permite realizar transacciones de forma sencilla y segura, tanto con tarjeta como con saldo en cuenta. Es muy popular entre los usuarios por su rapidez y confianza, aunque cobra comisiones algo más altas que otras alternativas.

Ventajas:

- Gran confianza y reconocimiento internacional.
- Permite pagar sin necesidad de introducir datos bancarios en cada compra.
- Interfaz sencilla para el usuario.
- Compatible con muchas plataformas de comercio electrónico.

Desventajas:

- Comisiones elevadas por transacción.
- Poca flexibilidad para personalizar el flujo de pago.
- Integración técnica menos avanzada que otras opciones.
- Posibles retenciones temporales de fondos en algunas operaciones.

Redsys

Redsys es la pasarela de pago más extendida en España y se utiliza en la mayoría de tiendas online que trabajan con bancos nacionales [16]. Ofrece un alto nivel de seguridad y cumple con las normativas europeas, aunque su configuración inicial puede ser más compleja y su documentación menos accesible para desarrolladores.

Ventajas:

- Muy utilizada en España, con soporte por parte de los principales bancos.

- Alta seguridad gracias al uso de sistemas 3D Secure.
- Cumple con las normativas PSD2 y SCA.
- Fiable y con buena reputación en entornos comerciales.

Desventajas:

- Configuración inicial compleja.
- Documentación técnica limitada.
- Poco orientada a desarrolladores.
- Limitada flexibilidad en integraciones personalizadas.

Stripe

Stripe es una pasarela de pago moderna pensada especialmente para desarrolladores [17]. Su API es clara y bien documentada, lo que facilita mucho la integración con aplicaciones web. Permite gestionar pagos con tarjeta, transferencias y otros métodos de forma segura y rápida. Es una opción muy utilizada por startups y empresas tecnológicas por su facilidad de uso y sus amplias posibilidades de personalización.

Ventajas:

- API muy completa y fácil de integrar.
- Admite múltiples métodos de pago y divisas.
- Buen equilibrio entre seguridad y personalización.
- Excelente documentación y comunidad activa.

Desventajas:

- Requiere conocimientos técnicos básicos para la configuración.
- Comisiones similares a las de PayPal.
- Depende de una conexión estable con la API.
- No disponible en todos los países.

Comparación y elección

PayPal, Redsys y Stripe son tres opciones válidas para integrar pagos en una tienda online. PayPal destaca por su popularidad y facilidad de uso, mientras que Redsys es la alternativa más común en España y ofrece gran seguridad. Sin embargo, para este TFG se ha optado por **Stripe** debido a su flexibilidad, buena documentación y facilidad de integración con tecnologías modernas como Spring Boot y Angular. Además, su API REST simplifica la conexión con el backend y garantiza un proceso de pago seguro y fluido para el usuario.

Criterio	PayPal	Redsys	Stripe
Popularidad	Muy alta	Alta	Alta
Facilidad de integración	Media	Baja	Alta
Comisiones	Altas	Medias	Medias
Orientación a desarrolladores	Baja	Baja	Muy alta
Seguridad	Alta	Muy alta	Alta

Cuadro 2.6: Comparativa de pasarelas de pago

Capítulo 3

Requisitos, especificaciones, coste, riesgos, viabilidad

En este capítulo se presenta un estudio inicial del proyecto, analizando los requisitos, especificaciones, costes, riesgos y viabilidad del sistema desarrollado. El objetivo es definir las **funcionalidades necesarias** para el correcto funcionamiento de la aplicación, detallar sus **especificaciones técnicas**, estimar el **coste y el tiempo de desarrollo**, identificar los **principales riesgos** y, finalmente, valorar la **viabilidad global** del proyecto. A continuación, se detallan los distintos apartados que conforman este estudio.

3.1. Requisitos

El sistema propuesto consiste en una **plataforma web de comercio electrónico para una droguería** que permite a los usuarios registrarse, explorar productos, filtrar resultados, gestionar su carrito de compra, realizar pagos mediante un servicio externo (Stripe) y recibir notificaciones automáticas mediante el bot de Telegram. Por parte del administrador, el sistema debe permitir filtrar pedidos, añadir productos, así como consultar y modificar los pedidos existentes.

A continuación, se detallan los requisitos funcionales y no funcionales definidos para el proyecto.

3.1.1. Requisitos funcionales

- **RF-01:** El sistema debe permitir el registro y autenticación de usuarios mediante correo electrónico y contraseña.
- **RF-02:** El sistema debe permitir que los usuarios consulten y filtren el catálogo de productos disponible.
- **RF-03:** El sistema debe permitir añadir, modificar o eliminar productos del carrito.
- **RF-04:** El sistema debe permitir realizar pedidos a través de una pasarela de pago (Stripe).
- **RF-05:** El sistema debe permitir al administrador gestionar el inventario de productos (crear, editar, eliminar).

- **RF-06:** El sistema debe generar y almacenar facturas de los pedidos realizados.
- **RF-07:** El sistema debe enviar notificaciones automáticas de confirmación de pedido y aviso de bajo stock mediante el bot de Telegram.
- **RF-08:** El sistema debe permitir consultar el historial de pedidos y su estado.
- **RF-09:** El sistema debe eliminar automáticamente los carritos inactivos tras un periodo determinado.

3.1.2. Requisitos no funcionales

- **RNF-01:** El sistema debe desarrollarse siguiendo una arquitectura cliente-servidor (Angular + Spring Boot + MySQL).
- **RNF-02:** La interfaz debe ser intuitiva, responsiva y accesible desde distintos dispositivos.
- **RNF-03:** El sistema debe garantizar la seguridad de los datos mediante cifrado de contraseñas y uso de HTTPS.
- **RNF-04:** La base de datos debe permitir transacciones seguras y mantener la integridad referencial.
- **RNF-05:** El código debe estar estructurado siguiendo buenas prácticas de desarrollo (modularización, uso de control de versiones).
- **RNF-06:** El sistema debe permitir su despliegue local o en la nube sin necesidad de licencias adicionales.
- **RNF-07:** Las notificaciones mediante el bot de Telegram deben ejecutarse en un entorno controlado y seguro.

3.2. Especificaciones

El sistema está diseñado como una **aplicación web moderna** basada en una arquitectura **frontend-backend**, en la que el cliente (Angular) se comunica con el servidor (Spring Boot) mediante *API REST*. La base de datos utilizada es **MySQL**, encargada de almacenar toda la información relativa a usuarios, productos, pedidos, pagos y facturas. Además, el sistema incorpora un **bot de Telegram** para gestionar las notificaciones automáticas postcompra y los avisos de bajo stock.

El sistema distingue entre dos tipos principales de usuarios:

- **Cliente:** puede registrarse, gestionar su carrito, realizar compras y consultar su historial de pedidos.
- **Administrador:** gestiona el catálogo de productos, controla los pedidos y supervisa las automatizaciones.

La aplicación está desarrollada íntegramente en **español**, es **multiplataforma** y accesible desde cualquier navegador moderno. Todos los componentes empleados son **gratuitos y de código abierto**, lo que facilita su instalación y mantenimiento sin coste adicional.

3.3. Tareas y costes

Para planificar el desarrollo del proyecto se ha elaborado una **Estructura de Descomposición del Trabajo (EDT)** que organiza las actividades en cinco fases principales: estudio, análisis, diseño, implementación y pruebas. Esta estructura permite identificar las dependencias entre tareas y establecer una secuencia lógica de ejecución.

3.3.1. Definición de tareas

ID	Nombre de la tarea	Dependencias
1	Estudio y especificación del proyecto	–
1.1	Definición de requisitos	–
1.2	Estimación de costes y planificación	1.1
1.3	Análisis de riesgos	1.2
1.4	Estudio de herramientas (Angular, Spring Boot, bot de Telegram)	1.3
2	Análisis del sistema	1
2.1	Casos de uso de la aplicación	1.4
2.2	Modelo de datos y entidades principales	2.1
3	Diseño del sistema	2
3.1	Arquitectura general (frontend-backend-bot de Telegram)	2.2
3.2	Diagramas UML y diseño de base de datos	3.1
4	Implementación	3
4.1	Desarrollo del backend en Spring Boot	3.2
4.2	Desarrollo del frontend en Angular	4.1
4.3	Integración con Stripe y MySQL	4.2
4.4	Configuración del bot de Telegram	4.3
5	Pruebas y validación	4
5.1	Pruebas unitarias e integración	4.4
5.2	Validación con usuarios y optimización final	5.1

Cuadro 3.1: Estructura de Descomposición del Trabajo (EDT) y dependencias entre tareas.

3.3.2. Estimación temporal

Para estimar la duración de cada fase se ha utilizado la técnica **PERT (Program Evaluation and Review Technique)**, que permite obtener una estimación temporal basada en tres valores: el tiempo optimista, el más probable y el pesimista. Esta metodología se fundamenta en los principios clásicos de estimación temporal descritos en la gestión de proyectos [18] y calcula el tiempo estimado (TE) mediante la fórmula:

$$TE = \frac{O + 4M + P}{6}$$

donde O es el tiempo optimista, M el más probable y P el pesimista.

Fase	O	M	P	TE (días)
Estudio y especificación	3	5	7	5
Análisis	4	6	8	6
Diseño	6	8	10	8
Implementación	20	25	30	25
Pruebas	6	8	10	8
Total estimado				52 días

Cuadro 3.2: Estimación temporal de las fases del proyecto mediante el método PERT.

Además de la técnica **PERT**, se ha aplicado el método de **juicio de expertos** con el objetivo de validar las estimaciones obtenidas y asegurar que los tiempos calculados sean realistas. Para ello, se han considerado las opiniones de tres perfiles con diferentes niveles de experiencia en el ámbito del desarrollo de software, lo que permitió ajustar las duraciones de las tareas y obtener una planificación más equilibrada:

- **Perfil 1 — Estudiante avanzado:** estudiante de último curso del Grado en Ingeniería Informática, con experiencia en proyectos académicos y conocimientos de programación web. Estimó que el proyecto podría completarse en torno a unos **55 días**, dedicando más tiempo a las fases de diseño e implementación, al considerar que estas suponen un mayor reto técnico en un proyecto individual.
- **Perfil 2 — Desarrollador junior:** profesional con unos dos años de experiencia en el desarrollo de aplicaciones web. Según su criterio, la duración total estaría alrededor de **50 días**, al poder abordar las fases de análisis e implementación con mayor agilidad, aunque manteniendo márgenes prudentes en las fases de pruebas y documentación.
- **Perfil 3 — Desarrollador senior/jefe de proyecto:** profesional con más de cinco años de experiencia en la gestión y supervisión de proyectos informáticos. Consideró que el proyecto podría completarse en torno a **48 días**, optimizando las fases de diseño e implementación gracias a una mejor planificación y experiencia previa en proyectos similares. Su punto de vista permitió validar la coherencia general del calendario y la estimación global del esfuerzo.

El contraste de opiniones entre estos perfiles permitió confirmar que las estimaciones iniciales eran adecuadas, situándose todas en un rango comprendido entre los 48 y 55 días, con una media aproximada de **50 días**. A partir de sus aportaciones, se fijó una duración final de **52 días**, incorporando un pequeño margen de contingencia que refleja un equilibrio entre las distintas perspectivas.

3.3.3. Diagrama de Gantt

El siguiente diagrama de Gantt muestra la planificación temporal del proyecto de manera visual, indicando la duración de cada fase y su relación con las tareas definidas en la estructura de descomposición del trabajo (EDT). Se ha generado a partir de las estimaciones realizadas mediante la técnica PERT y validadas con el juicio de expertos.

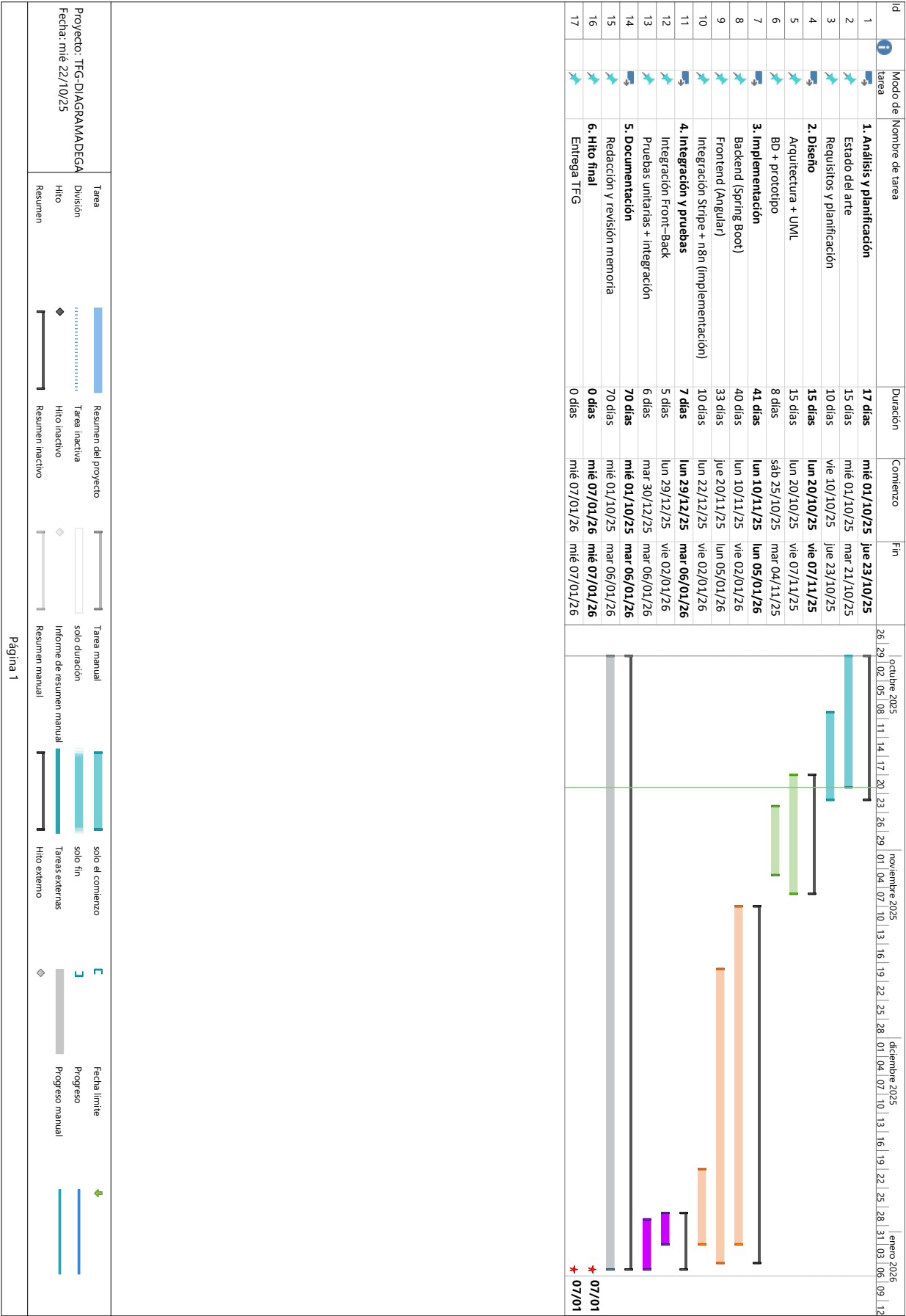


Figura 3.1.: Diagrama de Gantt del proyecto (01/10/2025–07/01/2026).

3.3.4. Estimación de costes

Para el cálculo de costes se han tenido en cuenta tanto los recursos humanos como los materiales y el software utilizado. Aunque el proyecto ha sido desarrollado por una sola persona, se ha desglosado el trabajo según distintos perfiles habituales en el desarrollo de software: jefe de proyecto, analista, desarrollador y tester. De esta forma se obtiene una estimación más ajustada y realista del esfuerzo requerido.

Perfil	Horas asignadas
Jefe de proyecto / Analista	40 h
Desarrollador backend	140 h
Desarrollador frontend	100 h
Tester / Validación	40 h
Total de horas estimadas	320 h

Cuadro 3.3: Reparto de horas por perfil utilizado para la estimación de costes.

Concepto	Detalle	Coste estimado
Coste de personal	Jefe de proyecto / analista: 40 h × 18 €/h	720 €
	Desarrollador backend: 140 h × 12 €/h	1.680 €
	Desarrollador frontend: 100 h × 12 €/h	1.200 €
	Tester / validación: 40 h × 10 €/h	400 €
Subtotal personal		4.000 €
Seguridad Social (30 %)	Aplicado al coste de personal (30 % × 4.000 €)	1.200 €
Coste de software	Herramientas gratuitas (Angular, Spring Boot, MySQL, VSCode, Postman)	0 €
Coste de software con licencia universitaria	Visual Paradigm (licencia académica proporcionada por la universidad)	0 €
Coste de hardware (amortizado)	Portátil personal (1.200 € de valor, amortizado 4 meses sobre 4 años)	100 €
Costes indirectos	Electricidad, Internet y agua durante el desarrollo (estimado)	80 €
Total estimado del proyecto		5.380 €

Cuadro 3.4: Estimación de costes detallada por perfil y recursos.

Nota: el valor del hardware se ha amortizado en base a una vida útil estimada de cuatro años.

Además, se incluyen las características del equipo utilizado durante el desarrollo, ya que impactan tanto en la amortización del hardware como en el rendimiento del proceso de trabajo:

- **CPU:** Intel Core i7-12700H
- **RAM:** 16 GB DDR5
- **Almacenamiento:** SSD NVMe 1 TB
- **Gráfica:** NVIDIA RTX 3050 Ti
- **Sistema operativo:** Windows 11

Para la planificación temporal se ha aplicado la técnica **PERT**, combinada con el **juicio de expertos**, comparando los valores calculados con estimaciones de proyectos similares y con referencias salariales obtenidas de portales como Indeed [?]. Además, se consultaron tres perfiles con diferente nivel de experiencia en el desarrollo de software, lo que permitió contrastar las horas estimadas y ajustar los tiempos de las tareas.

Finalmente, el coste total del proyecto se obtiene mediante la siguiente expresión:

$$C_{\text{total}} = C_{\text{personal}} + C_{\text{SeguridadSocial}} + C_{\text{hardware}} + C_{\text{software}} + C_{\text{indirectos}}$$

$$C_{\text{total}} = 4\,000 + 1\,200 + 100 + 0 + 80 = \mathbf{5\,380\,€}$$

El resultado confirma que el proyecto mantiene un coste razonable y acorde a su nivel de complejidad. El uso de herramientas libres y de una infraestructura propia reduce significativamente el coste total, manteniendo una buena relación entre esfuerzo, recursos y resultados obtenidos.

3.4. Riesgos

La gestión de riesgos permite anticipar posibles incidencias y definir estrategias preventivas que minimicen su impacto en el desarrollo del proyecto. En la siguiente tabla se resumen los principales riesgos identificados junto con su probabilidad, impacto y categoría general.

Riesgo identificado	Probabilidad	Impacto	Categoría
Problemas de integración entre módulos Angular–Spring Boot	Media	Alta	Técnica
Fallos en la pasarela de pago (Stripe)	Baja	Alta	Técnica
Cambios en los requisitos durante el desarrollo	Media	Media	Gestión
Sobrecarga de trabajo o retrasos en la planificación	Media	Media	Organización
Fallos de seguridad en la API o la base de datos	Baja	Alta	Seguridad
Falta de experiencia inicial con el bot de Telegram	Media	Media	Técnica

Cuadro 3.5: Principales riesgos identificados durante el desarrollo del proyecto.

A continuación, se describen las medidas generales de **mitigación y contingencia** aplicadas a los riesgos identificados:

- **Problemas de integración entre módulos Angular–Spring Boot:** se intentará detectar pronto cualquier error realizando pruebas cada vez que se conecten nuevas partes del sistema. También se mantendrá una buena organización del código para facilitar la comunicación entre ambos módulos.
- **Fallos en la pasarela de pago (Stripe):** antes de activar los pagos reales, se harán pruebas en el modo de simulación que ofrece la plataforma. En caso de error, se podrá reintentar el pago o registrar el pedido para revisarlo después.
- **Cambios en los requisitos durante el desarrollo:** se dejará algo de margen en la planificación para poder adaptarse a posibles cambios sin afectar al trabajo principal. Además, se priorizarán las funciones más importantes para tener siempre una versión funcional.
- **Sobrecarga de trabajo o retrasos en la planificación:** se dividirá el trabajo en tareas pequeñas y se intentará cumplir objetivos semanales. También se reservará algo de tiempo extra para posibles imprevistos o revisiones de última hora.
- **Fallos de seguridad en la aplicación o base de datos:** se harán copias de seguridad con frecuencia y se guardarán las contraseñas de forma segura. Además, se mantendrán las herramientas actualizadas para evitar problemas.
- **Falta de experiencia con el bot de Telegram:** se dedicará tiempo a aprender el funcionamiento de la librería antes de integrarlo por completo. Se probarán ejemplos sencillos para entender cómo gestionar comandos y notificaciones, y se consultará la documentación cuando sea necesario.

Gracias a estas medidas, se espera que los riesgos identificados tengan un impacto limitado en el desarrollo y no comprometan los objetivos principales del proyecto.

3.5. Viabilidad

3.5.1. Viabilidad legal

El proyecto no presenta impedimentos legales, ya que no maneja datos personales sensibles y cumple con la normativa vigente, especialmente con el Reglamento General de Protección de Datos (RGPD). Los pagos se realizan mediante la plataforma **Stripe**, que se encarga de procesar la información de forma segura y cifrada, evitando que el sistema tenga que almacenar datos de tarjetas. Todas las tecnologías empleadas son de **código abierto** o disponen de **licencias académicas gratuitas**, por lo que su uso es totalmente legal en el contexto de un proyecto universitario.

3.5.2. Viabilidad técnica

El uso conjunto de **Angular**, **Spring Boot**, **MySQL** y el **bot de Telegram** proporciona una solución estable, escalable y fácil de mantener. Son herramientas actuales, bien documentadas y con una amplia comunidad de desarrolladores, lo que facilita la resolución de problemas y reduce el riesgo de incompatibilidades. Además, el diseño modular del sistema permite incorporar futuras mejoras, como nuevos métodos de pago o notificaciones adicionales, sin necesidad de modificar la estructura principal del proyecto.

3.5.3. Viabilidad económica

El proyecto resulta **económicamente viable**, ya que su desarrollo no requiere inversión en licencias ni en infraestructura adicional. El software utilizado es **gratuito y de código abierto**, y el equipo empleado para el desarrollo es personal, por lo que los únicos costes reales corresponden al tiempo de dedicación y al uso del propio ordenador. Teniendo en cuenta que el coste total estimado del proyecto es de aproximadamente **5.380 €**, se puede concluir que se trata de una solución accesible, eficiente y sostenible para un entorno académico o de pequeña empresa.

Capítulo 4

Análisis

4.1. Diagramas de análisis

4.1.1. Diagrama de casos de uso

Para representar las interacciones entre los diferentes actores del sistema se ha elaborado el diagrama de casos de uso mostrado en la Figura 4.1. Este permite visualizar de forma general las funcionalidades principales y la relación entre el cliente, el administrador y el bot de Telegram.

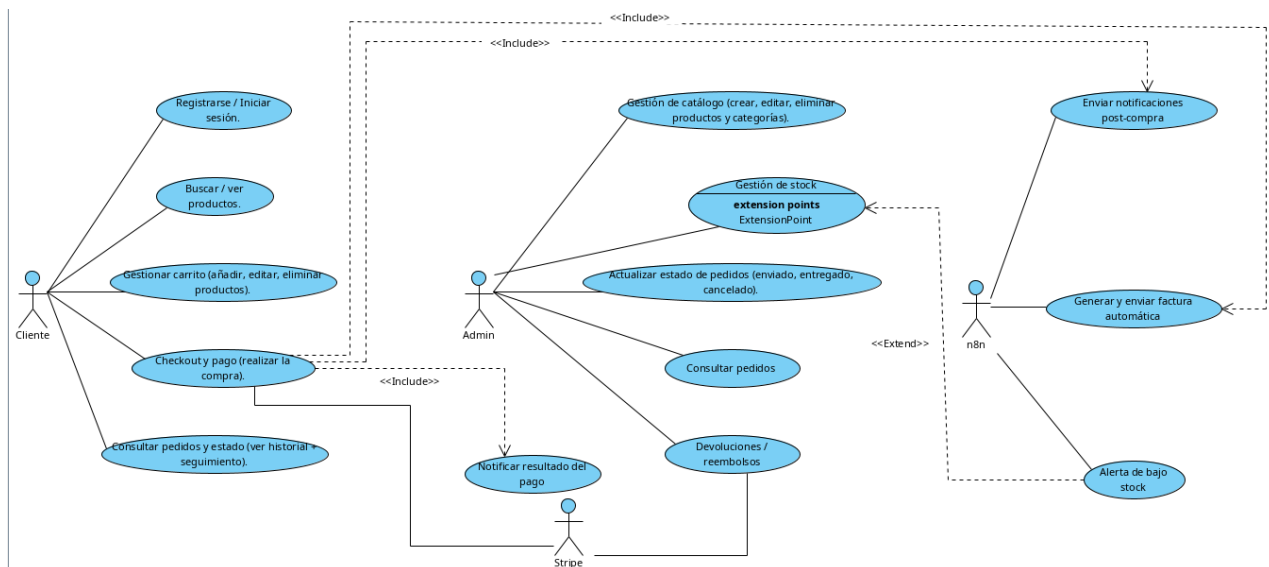


Figura 4.1: Diagrama de casos de uso del sistema.

4.1.2. Diagrama de clases de análisis

En la Figura 4.2 se muestra el **diagrama de clases de análisis** del sistema. Este modelo conceptual identifica las principales entidades que intervienen en el dominio de la aplicación, así como las relaciones que existen entre ellas. El cliente dispone de un carrito y puede realizar múltiples pedidos, cada uno de los cuales está asociado a un pago y a varios productos. Además, el cliente puede almacenar varias direcciones de envío, y los productos se agrupan dentro de categorías. El diagrama abstrae los detalles técnicos y se centra únicamente en los conceptos del negocio, sirviendo como base para el posterior diagrama de clases de diseño.

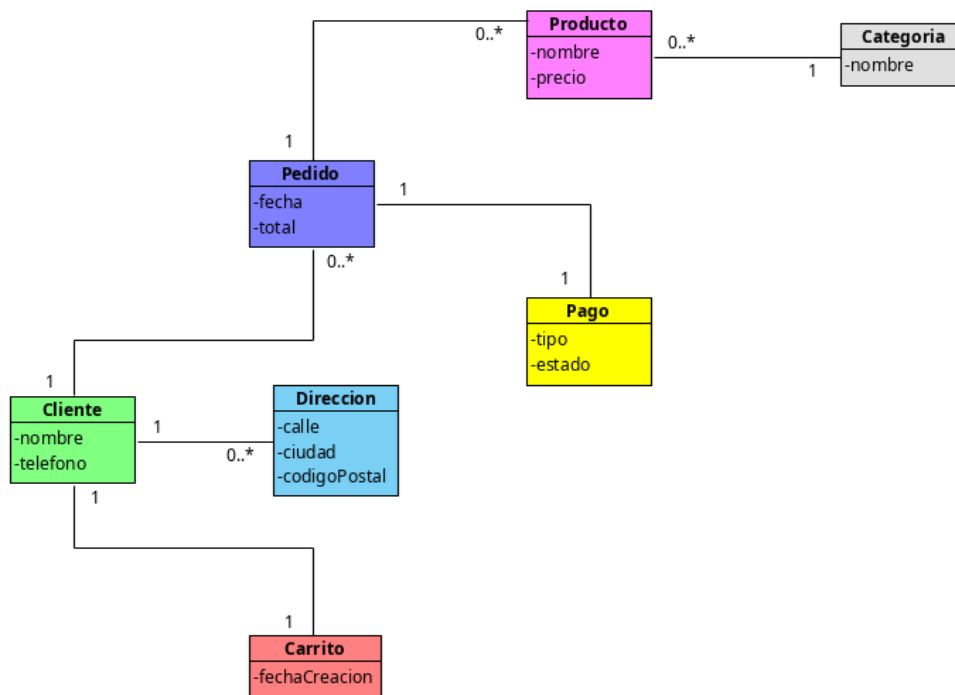


Figura 4.2: Diagrama de clases de análisis del sistema.

4.1.3. Diagrama de secuencia general del sistema (DSGS)

La Figura 4.3 muestra el **diagrama de secuencia general del sistema (DSGS)**, que representa de forma dinámica las interacciones entre los principales actores externos y la *WebApp*. En él se detalla el flujo completo de una compra, desde el inicio de sesión del cliente, la selección de productos y la realización del pedido, hasta la comunicación con la pasarela de pago *Stripe* y la notificación del resultado al *bot de Telegram*. El diagrama también incluye un flujo alternativo en el que se muestra el manejo de un posible error de pago. Este modelo complementa al diagrama de casos de uso al ofrecer una visión temporal del comportamiento global del sistema.

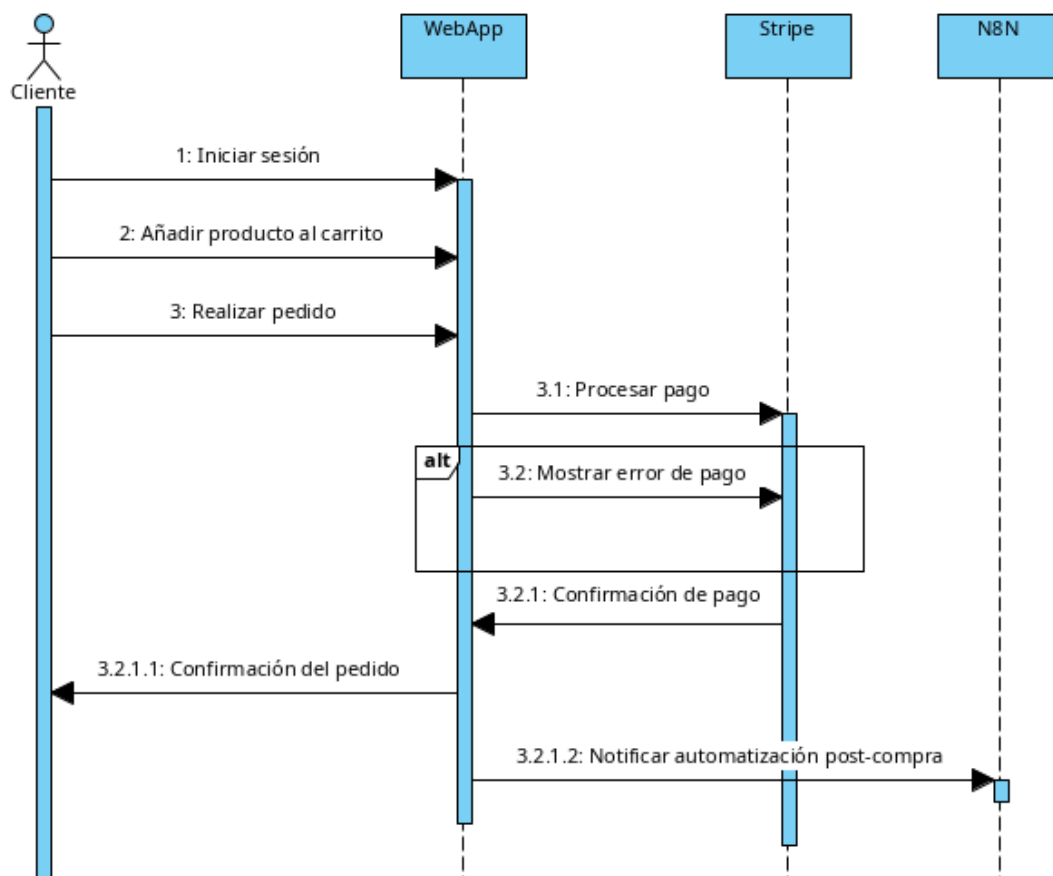


Figura 4.3: Diagrama de secuencia general del sistema (DSGS).

DSGS-00 Diagrama de secuencia general del sistema

Nombre	DSGS-00 Diagrama de secuencia general del sistema
Actor(es)	Cliente, WebApp, Stripe, Bot de Telegram
Descripción	Representa el flujo global del sistema durante el proceso de compra: inicio de sesión del cliente, gestión del carrito, procesamiento del pago a través de Stripe y notificación del resultado al bot de Telegram, incluyendo un flujo alternativo en caso de error de pago.
Precondiciones	El cliente debe tener acceso a la WebApp y existir conexión con Stripe y el bot de Telegram.
Postcondiciones	El pedido queda confirmado o rechazado, y el bot de Telegram recibe la notificación para ejecutar las tareas automáticas.
Flujo principal	1. El cliente inicia sesión. 2. Añade productos al carrito. 3. Realiza el pedido. 4. La WebApp envía el pago a Stripe. 5. Stripe confirma el pago. 6. La WebApp confirma el pedido. 7. La WebApp notifica al bot de Telegram para ejecutar tareas post-compra.
Flujos alternativos	4.a Stripe devuelve un error → la WebApp muestra el fallo al cliente.
Requisitos funcionales relacionados	RF-01, RF-02, RF-04, RF-06

Cuadro 4.1: Especificación del flujo DSGS-00 — Diagrama de secuencia del sistema.

Capítulo 5

Diseño

En este capítulo se presenta el proceso de diseño del sistema desarrollado, que abarca tanto la definición de la arquitectura general como los distintos diagramas UML que describen la estructura y el comportamiento de la aplicación. Finalmente, se incluye el diseño de la base de datos, que servirá como soporte a la implementación posterior.

5.1. Arquitectura general del sistema

El sistema se basa en una arquitectura de tres capas que facilita la separación de responsabilidades y la escalabilidad del proyecto:

- **Frontend:** desarrollado con un framework moderno Angular, encargado de la interacción con el usuario y la comunicación con el backend mediante peticiones HTTP.
- **Backend:** implementado con *Spring Boot*, que gestiona la lógica de negocio, las operaciones CRUD y la conexión con la base de datos.
- **Base de datos:** gestionada con el motor *MySQL*, donde se almacenan las entidades principales del sistema.

Además, el sistema integra un *bot de Telegram* encargado de tareas como el envío de notificaciones post-compra, la generación automática de facturas o la detección de carritos abandonados. Esta arquitectura modular permite una comunicación fluida entre componentes y facilita la futura ampliación del sistema.

5.2. Diagramas UML

Los diagramas UML se han elaborado con la herramienta *Visual Paradigm* para representar de forma visual los componentes y sus interacciones. Se incluyen diagramas de casos de uso, clases y secuencia.

5.2.1. Diagrama de clases

En la Figura 5.1 se muestra el diagrama de clases que define la estructura interna del sistema, las entidades principales y sus relaciones. Entre las clases más destacadas se

encuentran:

- **Usuario:** almacena la información del cliente o administrador.
- **Producto:** contiene los datos de los artículos disponibles.
- **Carrito y CarritoItems:** gestionan los productos añadidos por cada usuario.
- **Pedido, Pago y Factura:** representan el flujo de compra y facturación.

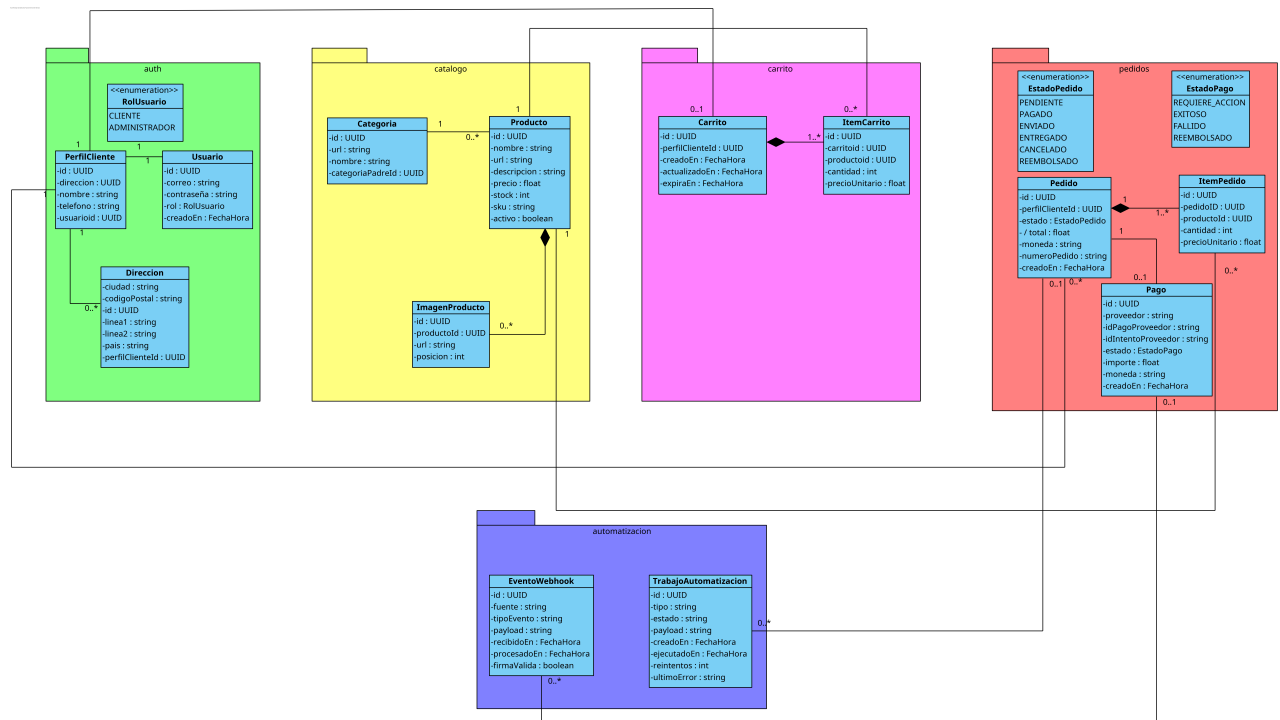


Figura 5.1: Diagrama de clases del sistema

5.2.2. Diagramas de secuencia

Los diagramas de secuencia reflejan los flujos de interacción entre los distintos componentes del sistema en diferentes escenarios funcionales. Estos esquemas permiten entender cómo se comunican los distintos servicios y agentes en cada proceso clave del sistema.

Gestión de carrito En este diagrama se representa el flujo de interacción entre el cliente, la aplicación web y los servicios del catálogo y del carrito. El usuario puede añadir, modificar o eliminar productos del carrito, y el sistema actualiza los datos en tiempo real mostrando la información correspondiente a cada producto.

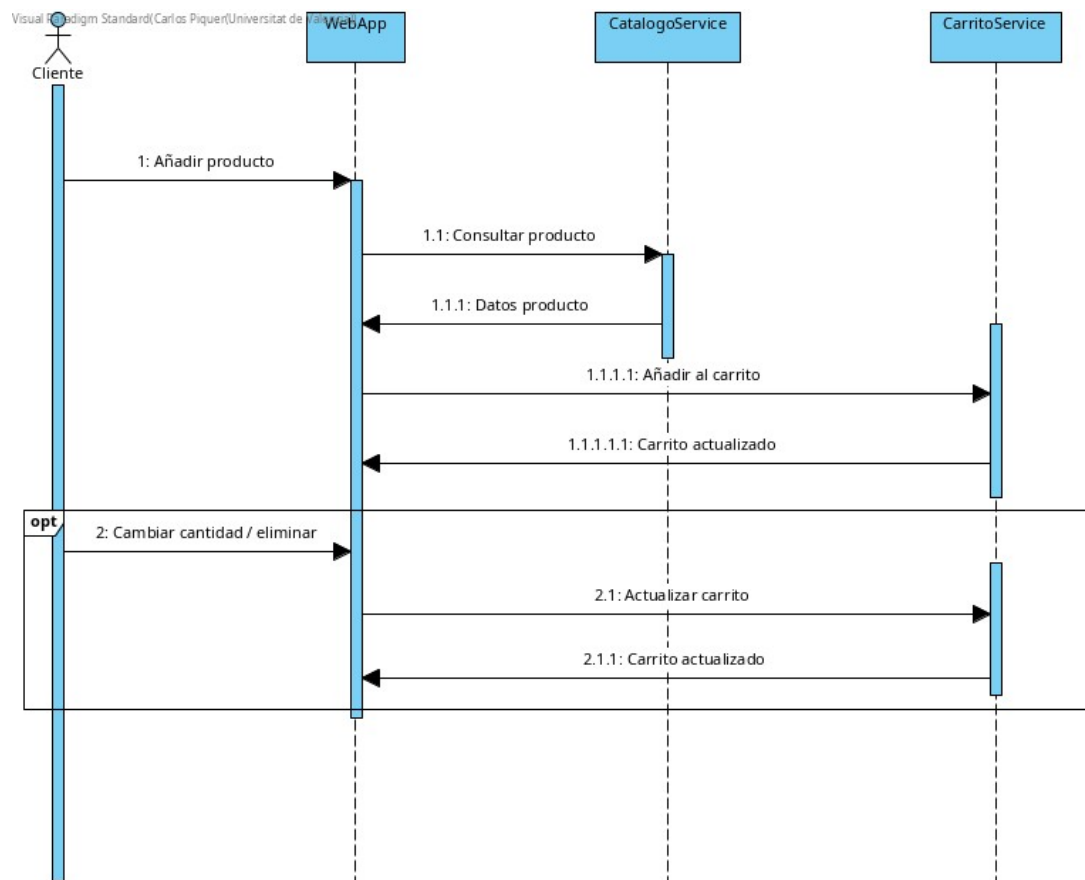


Figura 5.2: Diagrama de secuencia — gestión del carrito

DSGS-01 Gestión de carrito

Nombre	DSGS-01 Gestión de carrito
Actor(es)	Cliente, WebApp
Descripción	Representa el flujo general mediante el cual el cliente gestiona su carrito de compra: añadir productos, modificar cantidades o eliminar artículos, mientras la WebApp mantiene los datos sincronizados.
Precondiciones	El cliente debe disponer de acceso a la WebApp y el catálogo debe estar operativo.
Postcondiciones	El carrito queda actualizado correctamente con los productos seleccionados y sus cantidades.
Flujo principal	1. El cliente visualiza el catálogo. 2. Selecciona un producto y lo añade al carrito. 3. La WebApp actualiza las unidades y el total del carrito. 4. El cliente modifica cantidades o elimina elementos. 5. El sistema mantiene los datos sincronizados.
Flujos alternativos	2.a El producto no tiene suficiente stock y se muestra un aviso. 4.a Si el cliente no tiene carrito, el sistema crea uno automáticamente.
Requisitos funcionales relacionados	RF-02, RF-03

Cuadro 5.1: Especificación del flujo DSGS-01 Gestión de carrito.

Compra estándar A continuación, se muestra el flujo completo del proceso de compra. Incluye la autenticación del usuario, la creación del checkout, la comunicación con el servicio de pago *Stripe* y la confirmación del pedido. También se representa la respuesta automatizada del *bot de Telegram* para confirmar la compra, descontar el stock y generar la factura.

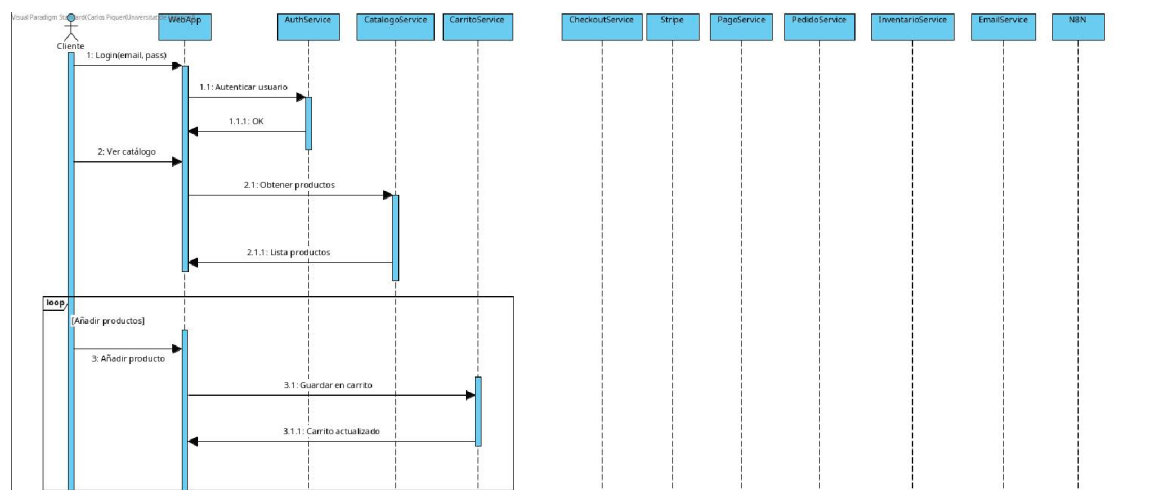


Figura 5.3: Diagrama de secuencia — parte 1: autenticación y carrito

DSGS-02 Compra estándar — Parte 1 (autenticación y carrito)

Nombre	DSGS-02 Compra estándar — Parte 1
Actor(es)	Cliente, WebApp
Descripción	Describe el inicio del proceso de compra: autenticación del cliente y revisión del carrito antes de continuar al checkout.
Precondiciones	El cliente debe estar registrado y tener acceso a la WebApp.
Postcondiciones	El cliente queda autenticado y el carrito listo para proceder al pago.
Flujo principal	1. El cliente accede a la WebApp. 2. El sistema solicita autenticación. 3. El cliente introduce sus credenciales. 4. La WebApp valida la autenticación. 5. El cliente revisa el carrito y continúa al checkout.
Flujos alternativos	4.a Credenciales incorrectas → se muestra un mensaje de error.
Requisitos funcionales relacionados	RF-01, RF-02

Cuadro 5.2: Especificación del flujo DSGS-02 Compra estándar — Parte 1.

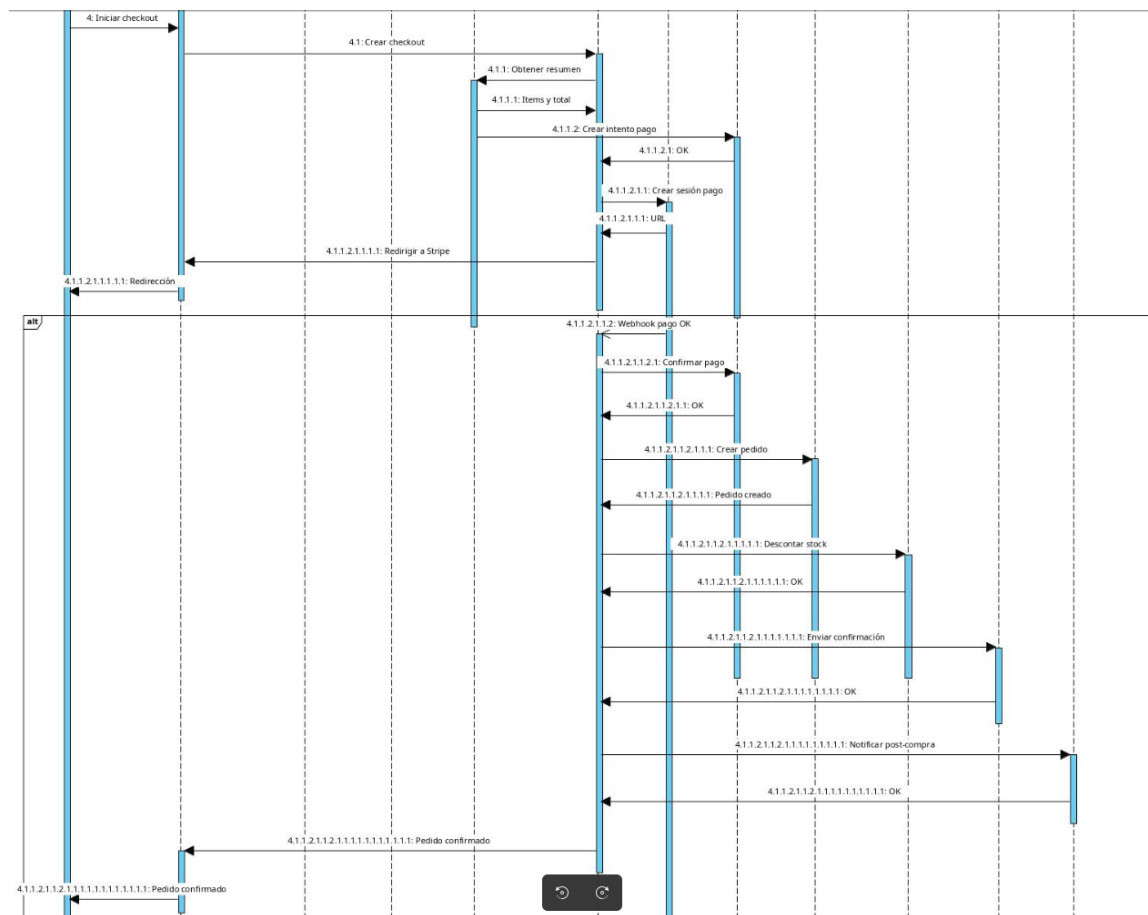


Figura 5.4: Diagrama de secuencia — parte 2: proceso de checkout y confirmación

DSGS-03 Compra estándar — Parte 2 (checkout y pago)

Nombre	DSGS-03 Compra estándar — Parte 2
Actor(es)	Cliente, WebApp, Stripe, Bot de Telegram
Descripción	Describe el proceso del checkout: introducción de datos, creación del pedido y procesamiento del pago mediante Stripe.
Precondiciones	El cliente debe estar autenticado y disponer de un carrito válido.
Postcondiciones	El pago se confirma, el pedido se marca como “pagado” y se notifica al bot de Telegram.
Flujo principal	1. El cliente accede al checkout. 2. Introduce datos personales y dirección. 3. Selecciona el método de pago. 4. La WebApp crea el pedido en estado “pendiente”. 5. Stripe procesa el pago. 6. Stripe confirma el pago a la WebApp. 7. La WebApp notifica al bot de Telegram para las tareas post-compra.
Flujos alternativos	5.a Stripe requiere autenticación adicional (3D Secure). 7.a Error notificando al bot de Telegram → reintento automático.
Requisitos funcionales relacionados	RF-04, RF-05, RF-06

Cuadro 5.3: Especificación del flujo DSGS-03 Compra estándar — Parte 2.

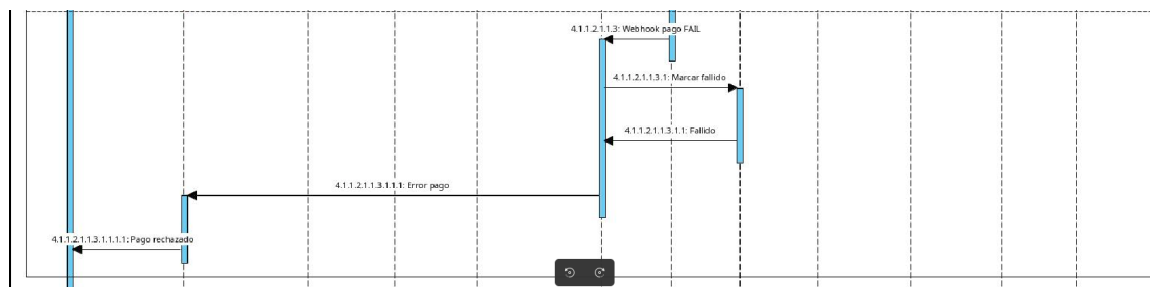


Figura 5.5: Diagrama de secuencia — parte 3: flujo alternativo de pago fallido

DSGS-04 Compra estándar — Parte 3 (pago fallido)

Nombre	DSGS-04 Compra estándar — Parte 3
Actor(es)	Cliente, WebApp, Stripe
Descripción	Representa el flujo alternativo donde el pago es rechazado por la pasarela de pago Stripe.
Precondiciones	El cliente se encuentra en el proceso de pago.
Postcondiciones	El pedido no se completa y se informa al cliente del error.
Flujo principal	1. Stripe intenta procesar el pago. 2. Stripe devuelve una respuesta de error. 3. La WebApp muestra un mensaje indicando que el pago ha fallado.
Flujos alternativos	— No aplica.
Requisitos funcionales relacionados	RF-04

Cuadro 5.4: Especificación del flujo DSGS-04 Compra estándar — Parte 3.

Generación de factura Este diagrama muestra el proceso automatizado de creación de facturas tras la confirmación de un pedido. Una vez validado el pago, el sistema genera el documento asociado y lo envía al cliente mediante el *bot de Telegram*, garantizando así un flujo de trabajo totalmente automatizado.

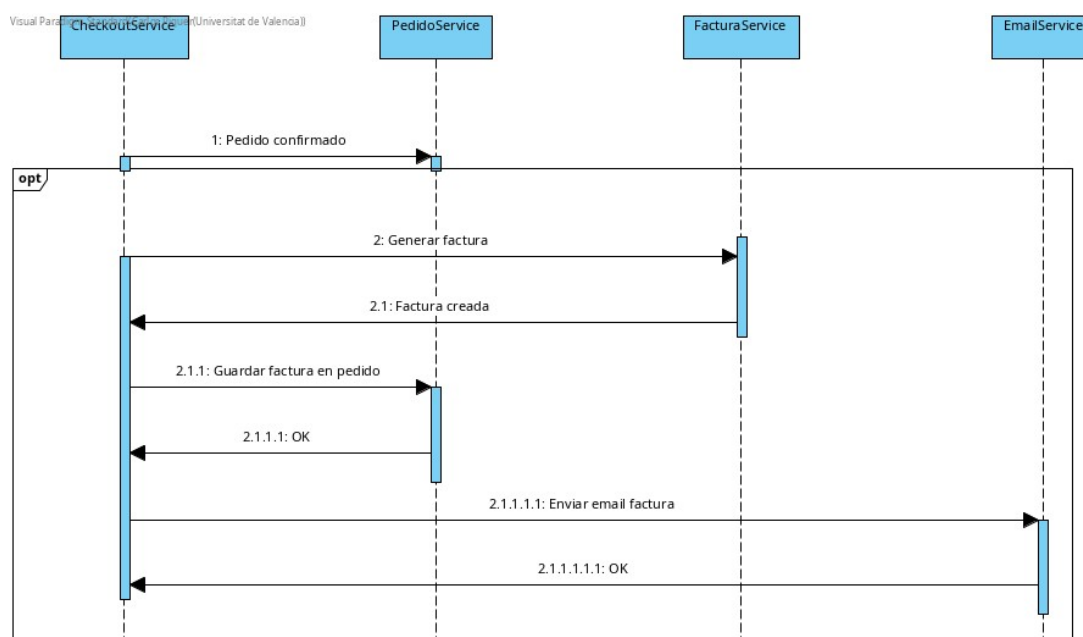


Figura 5.6: Diagrama de secuencia — generación de factura

DSGS-05 Generación de factura

Nombre	DSGS-05 Generación de factura
Actor(es)	WebApp, Bot de Telegram
Descripción	Una vez confirmado el pago del pedido, la WebApp notifica al bot de Telegram, que genera la factura correspondiente y la envía al cliente.
Precondiciones	El pedido debe estar registrado como pagado.
Postcondiciones	La factura queda creada, almacenada y enviada al cliente.
Flujo principal	1. La WebApp notifica al bot de Telegram indicando que el pedido está pagado. 2. El bot de Telegram genera la factura. 3. El bot de Telegram almacena o registra la factura en el sistema. 4. El bot de Telegram envía la factura por mensaje al cliente.
Flujos alternativos	2.a Error generando la factura → el bot de Telegram programa un reintento.
Requisitos funcionales relacionados	RF-06, RF-07

Cuadro 5.5: Especificación del flujo DSGS-05 Generación de factura.

Abandono de carrito Este diagrama refleja el funcionamiento del proceso de detección y eliminación de carritos inactivos. Si un usuario añade productos pero no completa la compra en un tiempo determinado, el sistema marca el carrito como abandonado y libera los productos reservados.

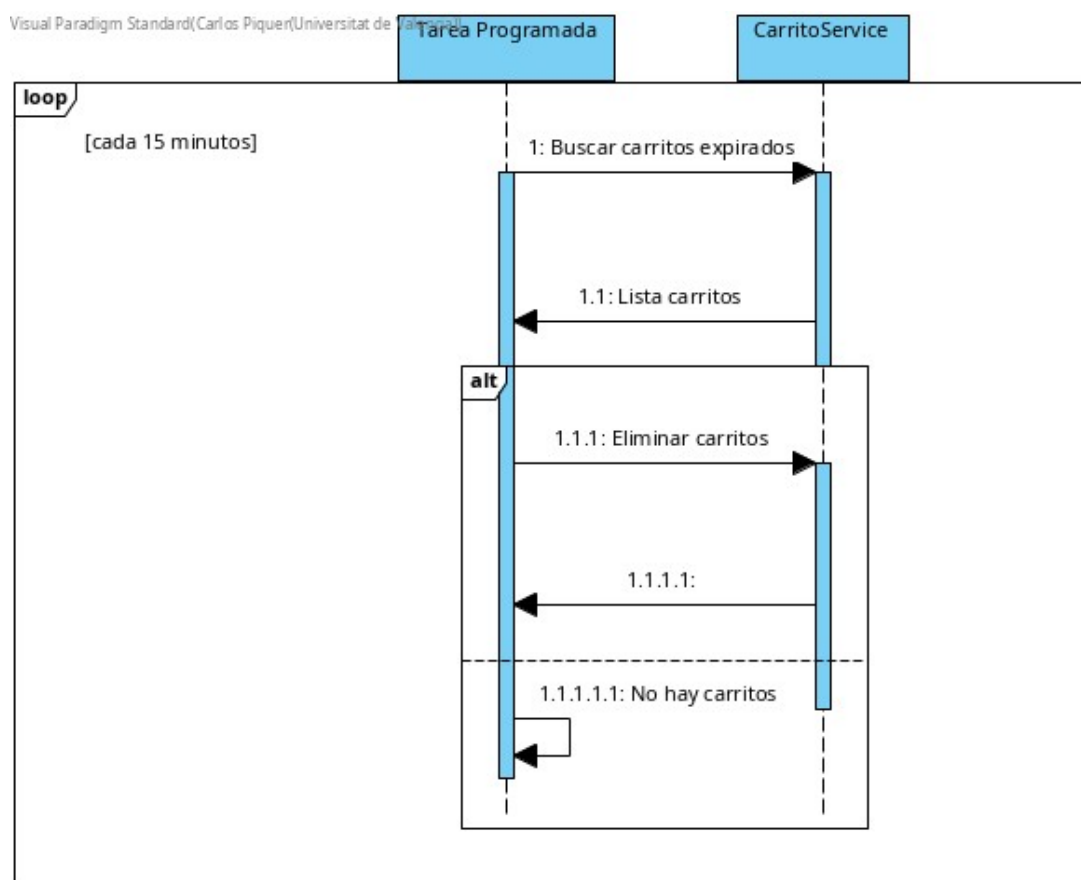


Figura 5.7: Diagrama de secuencia — abandono de carrito

DSGS-06 Abandono de carrito

Nombre	DSGS-06 Abandono de carrito
Actor(es)	WebApp, Bot de Telegram
Descripción	Proceso automatizado que detecta carritos inactivos y libera el stock asociado.
Precondiciones	Debe existir un carrito inactivo durante un periodo de tiempo determinado.
Postcondiciones	El carrito se marca como abandonado y el stock se libera.
Flujo principal	1. El bot de Telegram ejecuta una tarea programada. 2. Identifica los carritos inactivos. 3. Marca estos carritos como abandonados. 4. Libera stock en la base de datos.
Flujos alternativos	4.a Error actualizando stock → el bot de Telegram registra el error.
Requisitos funcionales relacionados	RF-09

Cuadro 5.6: Especificación del flujo DSGS-06 Abandono de carrito.

Bajo stock Por último, se representa el proceso de notificación de stock bajo. Cuando el inventario de un producto alcanza un umbral mínimo, el sistema ejecuta una tarea automatizada que envía un aviso al administrador mediante el *bot de Telegram*, permitiendo reponer el producto a tiempo.

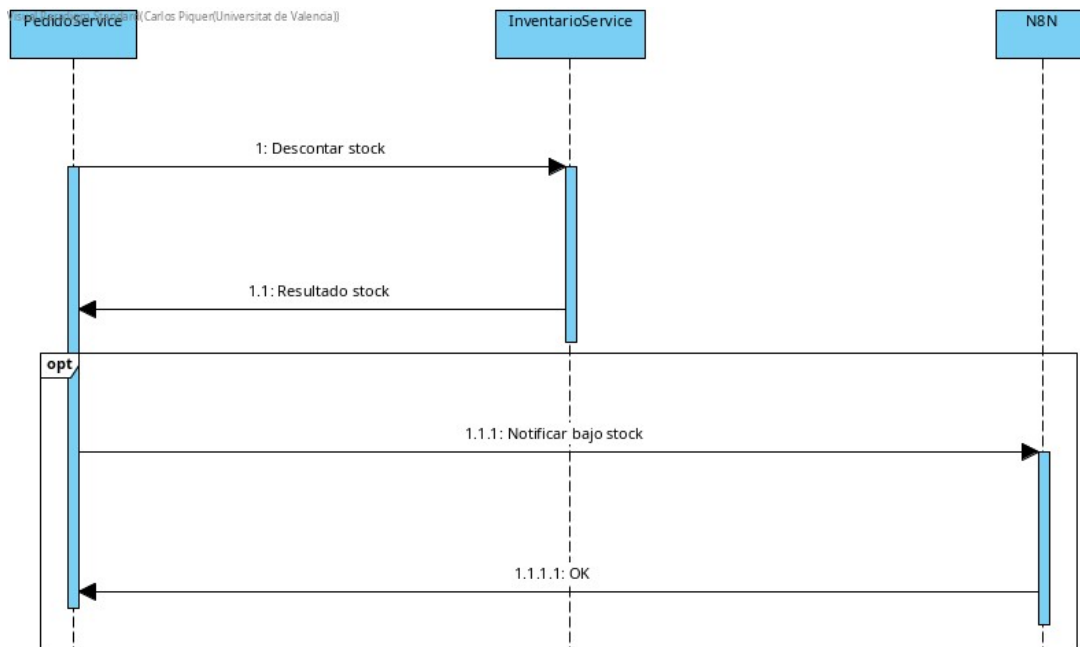


Figura 5.8: Diagrama de secuencia — notificación de bajo stock

DSGS-07 Bajo stock

Nombre	DSGS-07 Notificación de bajo stock
Actor(es)	WebApp, Bot de Telegram, Administrador
Descripción	Cuando el nivel de stock de un producto cae por debajo del umbral configurado, el bot de Telegram envía una notificación al administrador.
Precondiciones	Debe existir al menos un producto cuyo stock esté por debajo del mínimo.
Postcondiciones	El administrador recibe un aviso para reponer el stock.
Flujo principal	1. La WebApp detecta el cambio de stock. 2. Notifica al bot de Telegram. 3. El bot de Telegram verifica si el stock está por debajo del umbral. 4. El bot de Telegram envía una alerta al administrador.
Flujos alternativos	— No aplica.
Requisitos funcionales relacionados	RF-07

Cuadro 5.7: Especificación del flujo DSGS-07 Bajo stock.

5.3. Diseño de la base de datos

El diseño de la base de datos se llevó a cabo inicialmente mediante *dbdiagram.io*, para definir de manera visual las entidades, sus atributos y relaciones. Posteriormente, el modelo se validó e implementó en *MySQL Workbench*, comprobando su coherencia con el backend en *Spring Boot*.

5.3.1. Modelo entidad–relación

El modelo E/R del sistema representa las entidades principales (*usuarios*, *productos*, *carritos*, *pedidos*, *pagos*, *facturas*) y las relaciones entre ellas. Estas relaciones garantizan la integridad referencial mediante el uso de claves primarias, foráneas y restricciones de unicidad.

5.3.2. Modelo relacional y prototipo inicial

El modelo relacional resultante se compone de tablas como **usuarios**, **productos**, **carritos**, **carrito_items**, **pedidos**, **pagos** y **facturas**. La Figura 5.9 muestra el esquema final diseñado.

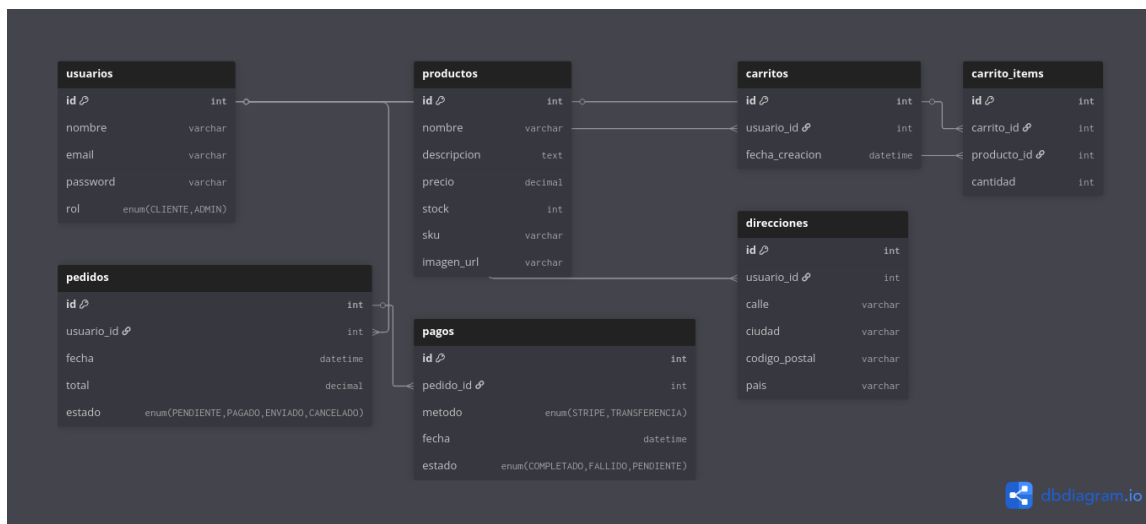


Figura 5.9: Prototipo de la base de datos relacional

5.4. Diseño de la interfaz de usuario

Además del diseño lógico y técnico del sistema, se ha desarrollado un prototipo visual que representa la estructura y apariencia de la aplicación web. Este diseño tiene como objetivo definir la disposición de los elementos de la interfaz, mejorar la experiencia de usuario y servir como guía en la fase de implementación del *frontend*.

5.4.1. Prototipo de la página de inicio

La Figura 5.10 muestra el prototipo correspondiente a la página principal de la aplicación, diseñado con la herramienta *Pencil Project*. La interfaz se ha elaborado siguiendo criterios de claridad, jerarquía visual y coherencia con el resto del sistema.

En la parte superior se encuentra la barra de navegación, que incluye el logotipo de la tienda, el buscador, las secciones principales y el acceso al carrito. A continuación, se muestra un banner promocional con un mensaje destacado y un botón de acción. Debajo se encuentran las categorías principales de productos, seguidas de una sección con artículos destacados y, finalmente, un pie de página con la información corporativa y enlaces a redes sociales.

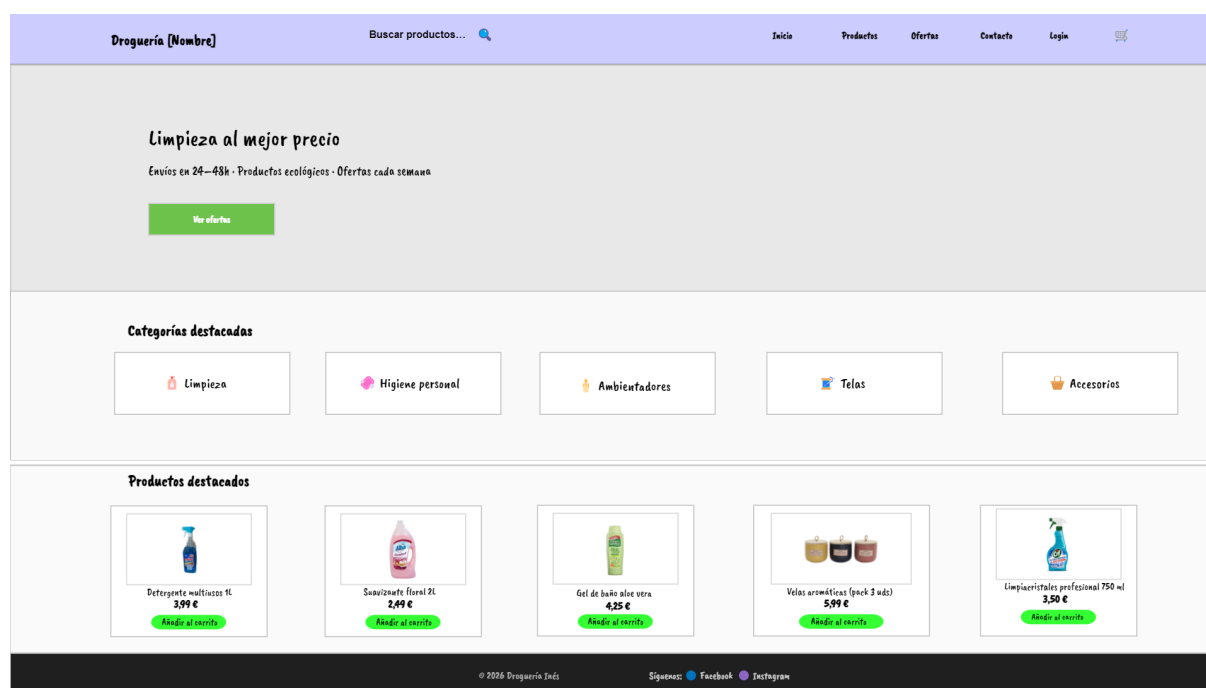


Figura 5.10: Prototipo de la página de inicio de la aplicación web.

5.4.2. Prototipo del catálogo de productos

La Figura 5.11 muestra el prototipo de la vista de catálogo, en la que el usuario puede explorar el inventario completo. La interfaz mantiene la coherencia visual con la página de inicio (mismo encabezado y pie de página) e incorpora una barra de filtros en la parte superior con búsqueda por texto, selección de categoría, rango de precio, opción de disponibilidad (*En stock*) y un control de *Ordenar por*. Bajo dicha barra se presenta una rejilla de productos (tarjetas con imagen, nombre, precio y botón *Añadir al carrito*) organizada en filas y columnas para facilitar el escaneo visual.

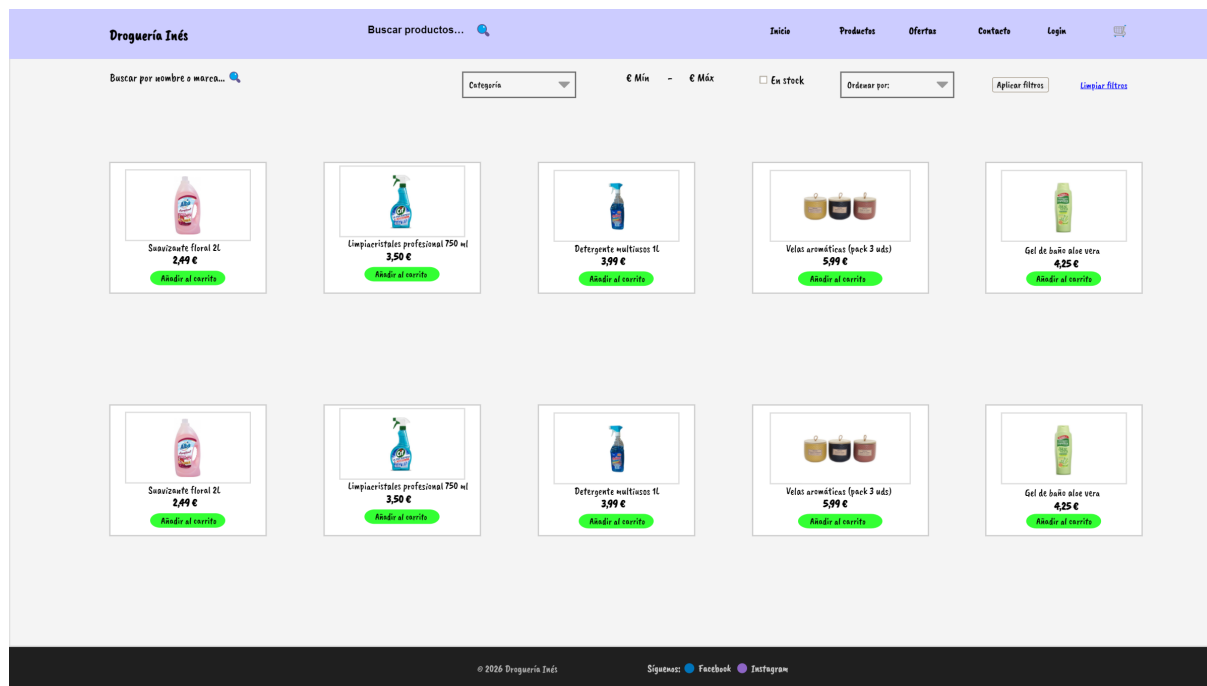


Figura 5.11: Prototipo del catálogo de productos con barra de filtros y rejilla de artículos.

5.4.3. Prototipo del carrito de compra

La Figura 5.12 muestra el prototipo de la vista de carrito. La interfaz mantiene la coherencia visual con el resto de pantallas (mismo encabezado y pie) y presenta una *tabla de líneas* donde se visualiza cada producto añadido con su imagen, nombre, precio unitario, selector de cantidad y subtotal, además de la acción de eliminar. En la parte inferior se destaca el importe **Total** y se incluyen los botones de navegación principales: *Seguir comprando* y *Finalizar compra*.

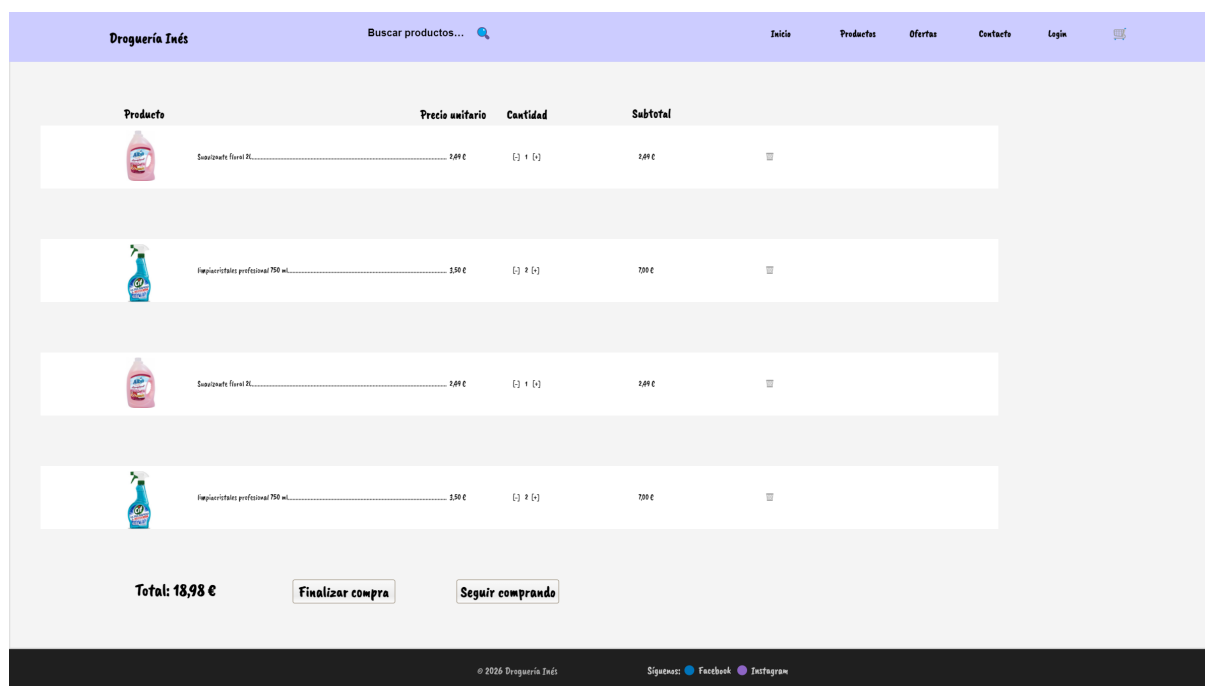


Figura 5.12: Prototipo del carrito de compra con líneas de producto, total y acciones principales.

5.4.4. Prototipo del proceso de pago (*Checkout*)

La Figura 5.13 muestra el prototipo correspondiente a la vista de pago del sistema. Esta pantalla representa la fase final del proceso de compra y mantiene la coherencia visual con el resto de interfaces, reutilizando la barra de navegación superior y el pie de página.

La interfaz se estructura en distintos bloques claramente diferenciados: **Datos personales**, **Dirección de envío**, **Método de pago** y **Tarjeta**. Cada sección incluye los campos necesarios para introducir la información correspondiente, priorizando la claridad y la usabilidad. En la parte inferior derecha se presenta el **resumen del pedido**, donde se muestran los productos seleccionados, el desglose de precios (subtotal, envío y total) y las acciones finales: *Volver al carrito* y *Confirmar pedido*.

La imagen muestra un prototipo de la interfaz de usuario para el proceso de pago (Checkout/Pago) de una aplicación llamada 'Droguería Inés'. La barra de navegación superior es de color morado y contiene el logo, un campo de búsqueda, y enlaces a Inicio, Productos, Ofertas, Contacto, Login y un icono de carrito. El título principal de la sección es 'Checkout / Pago' con el subtítulo 'Introduce tus datos para completar la compra.'.

El formulario está dividido en varias secciones:

- Datos personales:** Incluye campos para 'Nombre y apellidos', 'Correo electrónico' y 'Teléfono de contacto'.
- Dirección de envío:** Incluye campos para 'Dirección', 'Código postal', 'Ciudad', 'Provincia' y 'País'. Hay una opción desactivada 'Usar la misma dirección de facturación'.
- Método de pago:** Incluye radio buttons para 'Tarjeta de crédito / débito', 'PayPal', 'Transferencia bancaria' y 'En local'.
- Tarjeta:** Incluye campos para 'Número de tarjeta', 'Fecha de caducidad' y 'CVV'.
- Resumen del pedido:** Muestra una lista de productos con sus precios y cantidades. A la derecha, un resumen de precios: Subtotal: 9,49 €, Envío: 3,50 €, Total: 12,99 €. Al final del resumen hay dos botones: 'Volver al carrito' y 'Confirmar pedido'.

El pie de página es negro y contiene el copyright '© 2026 Droguería Inés', el texto 'Síguenos:' y los iconos de Facebook e Instagram.

Figura 5.13: Prototipo del proceso de pago con formularios y resumen del pedido.

Capítulo 6

Implementación y pruebas

6.1. Implementación

En este capítulo se describe el proceso de desarrollo de la aplicación, haciendo especial énfasis en los fragmentos de código más representativos del sistema. Se han seleccionado cuatro fragmentos que ilustran aspectos clave del desarrollo: la gestión del flujo de compra, la seguridad en la recuperación de contraseñas, la generación de estadísticas para el panel de administración y la integración con Cloudinary para la gestión de imágenes de productos.

6.1.1. Creación de pedido

El método `crearPedido`, ubicado en `PedidoController.java`, constituye el núcleo del flujo de compra del sistema. Se ejecuta dentro de una transacción `@Transactional`, lo que garantiza que, si se produce cualquier error durante el proceso —por ejemplo, stock insuficiente en alguno de los artículos—, todas las operaciones realizadas hasta ese momento se deshacen automáticamente, preservando la integridad de los datos.

Para cada producto del carrito, el sistema verifica la disponibilidad de stock antes de proceder al descuento. El fragmento siguiente muestra esta comprobación, junto con la lógica de actualización del inventario y el disparo automático de la alerta de bajo stock:

Listado 6.1: Verificación de stock y notificación al administrador (`PedidoController.java`).

```
// Verificar stock
if (productoReal.getStock() < item.getCantidad()) {
    throw new RuntimeException(
        "Stock insuficiente para: " + productoReal.getNombre());
}
// Restar stock y persistir
int nuevoStock = productoReal.getStock() - item.getCantidad();
productoReal.setStock(nuevoStock);
productoRepository.save(productoReal);
// Alerta Telegram si stock bajo
if (nuevoStock <= 5) {
    telegramService.notificarStockBajo(
        productoReal.getNombre(), nuevoStock);
}
```

Los cálculos monetarios se realizan con `BigDecimal` en lugar de tipos de coma flotante primitivos, evitando los errores de precisión habituales en operaciones aritméticas con decimales. Una vez completado el proceso, el sistema dispara dos notificaciones externas de forma automática: un correo electrónico de confirmación al cliente y un mensaje al administrador a través del bot de Telegram. Cabe destacar que el estado inicial del pedido depende del método de pago seleccionado: los pagos con Bizum arrancan en estado `PENDIENTE_PAGO` hasta que el administrador confirma la transferencia de forma manual.

6.1.2. Recuperación de contraseña

El módulo de recuperación de contraseña, implementado en `PasswordController.java`, sigue un flujo en dos pasos diseñado con criterios de seguridad. En el primer paso, el sistema genera un token único mediante `UUID.randomUUID()` y lo almacena en la base de datos junto con una marca de caducidad de una hora. A continuación, envía al usuario un enlace de recuperación por correo electrónico. En el segundo paso, el sistema valida el token recibido, comprueba que no haya expirado y actualiza la contraseña con el nuevo valor proporcionado.

Listado 6.2: Generación de token y validación en el reset de contraseña (`PasswordController.java`).

```
// PASO 1    Generar token con caducidad de 1 hora
String token = UUID.randomUUID().toString();
usuario.setResetToken(token);
usuario.setTokenExpiration(LocalDate.now().plusHours(1));
usuarioRepository.save(usuario);
emailService.sendPasswordResetEmail(email, usuario.getNombre(), token);

// Respuesta ambigua intencionada (evita user enumeration)
return "Si el correo existe, se ha enviado un enlace.";

// PASO 2    Comprobar caducidad y actualizar contraseña
if (usuario.getTokenExpiration().isBefore(LocalDate.now())) {
    return "Error: El enlace ha caducado.";
}
usuario.setPassword(passwordEncoder.encode(newPassword));
usuario.setResetToken(null);
usuario.setTokenExpiration(null);
```

Merece especial mención el mensaje de respuesta del primer paso: la formulación *"Si el correo existe, se ha enviado un enlace"* es intencionalmente ambigua. Esta técnica evita la enumeración de usuarios (*user enumeration*), impidiendo que un atacante pueda deducir qué direcciones de correo están registradas en el sistema a partir de las respuestas del servidor. Tras completar el reset, el token se elimina de la base de datos para que no pueda reutilizarse, y la nueva contraseña se almacena hasheada con *BCrypt*, garantizando que nunca se persista en texto plano.

6.1.3. Estadísticas del panel de administración

El endpoint `getStats`, definido en `StatsController.java`, centraliza en una única llamada todos los datos que alimentan el dashboard del administrador, consolidando información procedente de cuatro repositorios distintos: pedidos, productos, detalles de pedido y usuarios.

Listado 6.3: Cálculo de ingresos y métricas de stock en el panel de administración (`StatsController.java`).

```
// Ingresos del mes actual vs. mes anterior
LocalDateTime inicioMes =
    LocalDate.now().withDayOfMonth(1).atStartOfDay();
BigDecimal ingresosMes =
    pedidoRepository.sumIngresosPorPeriodo(
        inicioMes, inicioMes.plusMonths(1));
BigDecimal ingresosMesAnterior =
    pedidoRepository.sumIngresosPorPeriodo(
        inicioMes.minusMonths(1), inicioMes);

// Productos con stock crítico
List<Producto> stockCritico =
    productoRepository
        .findByStockLessThanEqualOrderByStockAsc(5);
long sinStock =
    stockCritico.stream()
        .filter(p -> p.getStock() == 0).count();

// Top 5 productos más vendidos
for (Object[] row : detallePedidoRepository
    .findTopProductos(PageRequest.of(0, 5))) { ... }
```

La comparativa de ingresos entre el mes actual y el mes anterior permite al administrador detectar tendencias de ventas de forma inmediata. El cálculo excluye los pedidos cancelados y los pendientes de pago, de modo que el valor mostrado refleja únicamente ingresos reales confirmados. En cuanto al inventario, el panel diferencia entre productos sin stock (`stock = 0`) y productos con stock bajo (`stock ≤ 5`), proporcionando dos niveles de alerta independientes. El listado de los cinco productos más vendidos se obtiene mediante una consulta JPQL con agregación y paginación controlada con `PageRequest`, limitando el resultado a los primeros cinco registros.

6.1.4. Gestión de imágenes con Cloudinary

La subida de imágenes de producto se gestiona a través de *Cloudinary*, un servicio de almacenamiento y distribución de contenido multimedia en la nube. Esta integración permite al administrador o al empleado añadir y actualizar las fotografías del catálogo desde cualquier dispositivo con acceso al panel de administración, sin necesidad de intervención técnica ni acceso al servidor.

El flujo de subida sigue una arquitectura en la que el frontend no se comunica directamente con Cloudinary, sino que envía el archivo al backend, que actúa como intermediario. De esta forma, las credenciales de la cuenta de Cloudinary permanecen en el servidor y

nunca quedan expuestas al cliente. El proceso completo es el siguiente: el administrador selecciona una imagen en el formulario del panel; Angular envía el archivo mediante una petición POST a `/api/productos/upload` en formato *multipart/form-data*; Spring Boot recibe el fichero, lo sube a Cloudinary en la carpeta **drogueria** usando el SDK oficial de Java, y retorna la URL pública generada; Angular almacena esa URL en el objeto del producto, que se persiste en la base de datos al guardar el formulario. A partir de ese momento, el catálogo carga las imágenes directamente desde la CDN de Cloudinary.

El fragmento siguiente corresponde al endpoint del backend responsable de la subida:

Listado 6.4: Endpoint de subida de imágenes a Cloudinary (ProductoController.java).

```
@PostMapping("/upload")
public ResponseEntity<?> subirImagen(
    @RequestParam("file") MultipartFile file) {
    try {
        Map uploadResult = cloudinary.uploader().upload(
            file.getBytes(),
            ObjectUtils.asMap("folder", "drogueria")
        );
        String url = (String) uploadResult.get("url");
        return ResponseEntity.ok()
            .body("{\"url\": \"" + url + "\"}");
    } catch (Exception e) {
        return ResponseEntity.badRequest()
            .body("Error subida: " + e.getMessage());
    }
}
```

En el lado del frontend, el método `onFileSelected` de `admin.component.ts` lanza la subida en cuanto el administrador selecciona el archivo, sin esperar a que se guarde el formulario del producto, lo que proporciona una respuesta inmediata en la interfaz:

Listado 6.5: Gestión de la selección de imagen en el panel de administración (`admin.component.ts`).

```
onFileSelected(event: any) {
    const file: File = event.target.files[0];
    if (file) {
        this.subiendoImagen = true;
        this.productoService.uploadImage(file).subscribe({
            next: (response: any) => {
                this.nuevoProducto.urlImagen = response.url;
                this.subiendoImagen = false;
            },
            error: () => { this.subiendoImagen = false; },
        });
    }
}
```

6.2. Pruebas funcionales

Las pruebas funcionales tienen como objetivo verificar que cada requisito funcional del sistema se satisface correctamente bajo condiciones reales de uso. Para ello, se han definido casos de prueba agrupados por bloque funcional, ejecutados de forma manual sobre la aplicación desplegada en local. Cada caso recoge la acción realizada, el resultado esperado y el estado final de la prueba.

6.2.1. Registro y autenticación (RF-01)

Este bloque valida el módulo de gestión de usuarios: el registro de nuevas cuentas, el inicio de sesión y la recuperación de contraseña. Las pruebas comprueban tanto el flujo correcto como la respuesta del sistema ante entradas inválidas o duplicadas.

ID	Caso de prueba	Resultado esperado	Estado
PF-01	Registro con datos válidos	Usuario creado y redirección al inicio	OK
PF-02	Registro con email ya existente	Mensaje de error	OK
PF-03	Login con credenciales correctas	Acceso a la cuenta	OK
PF-04	Login con contraseña incorrecta	Mensaje de error	OK
PF-05	Recuperación de contraseña con email válido	Envío de correo con enlace de recuperación	OK

Cuadro 6.1: Pruebas funcionales — Registro y autenticación (RF-01).

6.2.2. Catálogo y carrito (RF-02, RF-03)

Este bloque cubre la navegación por el catálogo de productos y la gestión del carrito de compra. Se comprueba que los filtros y la búsqueda devuelven resultados coherentes, y que las operaciones sobre el carrito —añadir, modificar cantidad y eliminar— se reflejan correctamente en la interfaz.

ID	Caso de prueba	Resultado esperado	Estado
PF-06	Filtrar productos por categoría	Solo se muestran los productos de esa categoría	OK
PF-07	Buscar producto por nombre	El producto aparece en los resultados	OK
PF-08	Añadir producto al carrito	El contador del carrito se incrementa	OK
PF-09	Modificar cantidad de un producto en el carrito	El total se actualiza	OK
PF-10	Eliminar producto del carrito	El producto desaparece de la lista	OK

Cuadro 6.2: Pruebas funcionales — Catálogo y carrito (RF-02, RF-03).

6.2.3. Proceso de compra (RF-04, RF-06)

Este bloque verifica el flujo completo de compra, incluyendo el pago mediante Stripe y la generación de facturas. Asimismo, se comprueba que el sistema reacciona correctamente ante un pago rechazado y que el stock se descuenta de forma apropiada tras confirmar el pedido.

ID	Caso de prueba	Resultado esperado	Estado
PF-11	Completar compra con tarjeta Stripe válida	Pedido confirmado y factura generada	OK
PF-12	Intentar compra con tarjeta rechazada	Mensaje de error de pago	OK
PF-13	Verificar descuento de stock tras la compra	El stock del producto disminuye en la cantidad pedida	OK

Cuadro 6.3: Pruebas funcionales — Proceso de compra (RF-04, RF-06).

6.2.4. Administrador (RF-05, RF-08)

Este bloque valida las funcionalidades reservadas al administrador: la gestión del catálogo de productos y la consulta del historial de pedidos. Se comprueba que las operaciones de creación, edición y eliminación se reflejan de forma inmediata en el catálogo, y que el listado de pedidos muestra la información y los estados correctamente.

ID	Caso de prueba	Resultado esperado	Estado
PF-14	Crear producto nuevo	El producto aparece en el catálogo	OK
PF-15	Editar precio de un producto	El precio se actualiza en el catálogo	OK
PF-16	Eliminar producto	El producto desaparece del catálogo	OK
PF-17	Ver historial de pedidos	Lista de pedidos con estados correctos	OK

Cuadro 6.4: Pruebas funcionales — Administrador (RF-05, RF-08).

6.2.5. Bot de Telegram (RF-07, RF-09)

Este bloque verifica las automatizaciones gestionadas por el bot de Telegram: la notificación de nuevos pedidos, el aviso de bajo stock y la eliminación automática de carritos inactivos. Durante las pruebas se detectó una incidencia en la alerta de bajo stock: inicialmente, la notificación solo se disparaba cuando el stock descendía como consecuencia de un pedido, pero no cuando el administrador modificaba el stock manualmente desde el panel de administración. Dicha incidencia fue corregida, de modo que la alerta se genera en ambos escenarios.

ID	Caso de prueba	Resultado esperado	Estado
PF-18	Completar una compra	Notificación de nuevo pedido al administrador por Telegram	OK
PF-19	Bajar stock a ≤ 5 mediante un pedido	Alerta de bajo stock al administrador por Telegram	OK
PF-20	Bajar stock a ≤ 5 desde el panel de administración	Alerta de bajo stock al administrador por Telegram	OK
PF-21	Dejar un carrito sin completar durante el tiempo configurado	El carrito se elimina automáticamente	OK

Cuadro 6.5: Pruebas funcionales — Bot de Telegram (RF-07, RF-09).

6.3. Pruebas de rendimiento

Las pruebas de rendimiento tienen como objetivo evaluar la velocidad de carga del frontend y los tiempos de respuesta del backend bajo condiciones de uso normal. Para ello se han empleado dos herramientas: *Google Lighthouse*, integrada en Chrome DevTools, para analizar el rendimiento de las páginas del frontend; y *Postman*, para medir los tiempos de respuesta de los principales endpoints de la API REST.

6.3.1. Rendimiento del frontend — Google Lighthouse

Las pruebas de Lighthouse se han ejecutado en modo escritorio (*desktop*) sobre el **build de producción** (`ng build`) de las páginas principales de la aplicación. Las métricas registradas son el índice de rendimiento general (*Performance*), el tiempo hasta el primer renderizado de contenido (*FCP*), el tiempo hasta el renderizado del elemento más grande (*LCP*) y el tiempo de bloqueo total (*TBT*).

Página	Performance	FCP	LCP	TBT
Página de inicio	89	0,7 s	2,1 s	40 ms
Catálogo de productos	93	0,8 s	1,6 s	60 ms
Carrito de compra	92	0,7 s	1,7 s	110 ms
Panel de administración	92	0,8 s	1,1 s	180 ms

Cuadro 6.6: Resultados de Lighthouse (build de producción).

Las cuatro páginas principales obtienen una puntuación de rendimiento igual o superior a 89, con tiempos LCP por debajo de 2,1 s en todos los casos. Durante las pruebas del catálogo se detectó que las imágenes de los productos penalizaban significativamente el LCP. Para corregirlo se aplicó *lazy loading* mediante el atributo `loading="lazy"` y se añadieron dimensiones explícitas (`width` y `height`) en las tarjetas de producto, lo que redujo el LCP del catálogo de 2,9 s a 1,6 s y elevó su puntuación de 82 a 93.

6.3.2. Rendimiento del backend — Tiempos de respuesta de la API

Los tiempos de respuesta de los principales endpoints del backend se han medido con *Postman* sobre el servidor local (<http://localhost:8080>). Cada petición se ha ejecutado con el sistema en estado de uso normal, con datos reales cargados en la base de datos.

Método	Endpoint	Tiempo de respuesta
GET	/api/productos	37 ms
GET	/api/productos/{id}	14 ms
GET	/api/stats	43 ms
GET	/api/pedidos/usuario/{id}	27 ms
POST	/api/pedidos/crear	1.870 ms

Cuadro 6.7: Tiempos de respuesta de los principales endpoints de la API REST.

Los cuatro endpoints de consulta responden en menos de 50 ms, resultados que se consideran óptimos para una API REST en entorno local. El endpoint `POST /api/pedidos/crear` presenta un tiempo notablemente superior (1.870 ms) debido a que ejecuta de forma síncrona varias operaciones encadenadas: la validación de usuario y stock, la escritura del pedido y sus detalles en la base de datos, el envío del correo de confirmación al cliente y la notificación al administrador mediante el bot de Telegram. En un entorno de producción real, estas operaciones secundarias se procesarían de forma **asíncrona** para reducir el tiempo de respuesta percibido por el usuario.

6.4. Pruebas de usabilidad

Las pruebas de usabilidad se han llevado a cabo mediante el *System Usability Scale* (SUS), un cuestionario estándar compuesto por 10 ítems con escala Likert del 1 al 5. La puntuación final, entre 0 y 100, se obtiene sumando $(x_i - 1)$ para los ítems impares y $(5 - x_i)$ para los pares, y multiplicando el resultado por 2,5. Según los baremos establecidos por Bangor, Kortum y Miller [?], puntuaciones superiores a 68 se consideran aceptables y superiores a 80, excelentes.

6.4.1. Participantes y tareas

Se seleccionaron cuatro participantes con perfiles diferenciados para cubrir tanto el público objetivo de la tienda como el perfil de administración:

Usuario	Perfil	Edad	Nivel digital
U1	Consumidor habitual de e-commerce	24	Alto
U2	Propietaria de la tienda (cliente objetivo)	45	Medio
U3	Usuaría con dificultades informáticas	55	Bajo
U4	Graduado en Ingeniería Informática	24	Experto

Cuadro 6.8: Perfiles de los participantes en las pruebas de usabilidad.

Cada participante realizó de forma autónoma las siguientes cinco tareas, sin instrucciones previas sobre el uso de la aplicación:

1. Registrarse como nuevo usuario en la plataforma.
2. Localizar un producto del catálogo y añadirlo al carrito.
3. Modificar la cantidad de un artículo en el carrito.
4. Completar el proceso de compra mediante la pasarela de pago Stripe.
5. Consultar el historial de pedidos en el área personal.

6.4.2. Resultados

La tabla 6.9 recoge las respuestas individuales de cada participante y la puntuación SUS resultante.

Usuario	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	SUS
U1 (24 a., consumidor)	4	2	4	1	4	2	5	2	4	1	82,5
U2 (45 a., propietaria)	5	1	4	2	4	1	4	1	5	2	87,5
U3 (55 a., baja competencia)	3	3	3	3	4	2	4	3	4	2	62,5
U4 (24 a., técnico)	4	1	5	1	5	1	5	1	5	1	97,5
Puntuación media											82,5

Cuadro 6.9: Respuestas individuales y puntuaciones SUS.

La puntuación media obtenida es de **82,5 sobre 100**, lo que sitúa al sistema en la categoría de **excelente**.

El usuario técnico (U4, 97,5) valoró positivamente la consistencia de la navegación y la correcta persistencia del estado de sesión. La propietaria de la tienda (U2, 87,5) encontró el panel de administración intuitivo y destacó la claridad en la gestión de pedidos, que es el uso que hará de la aplicación en el día a día. La usuaria con menor competencia digital (U3, 62,5) se situó por debajo del umbral de aceptable (68); la fricción se concentró en el paso del *checkout*, donde el recelo habitual ante formularios de pago requirió una breve explicación inicial. No obstante, una vez superado ese paso, completó el resto de las tareas de forma autónoma gracias a la claridad visual de los formularios. Este resultado apunta a una posible mejora futura: añadir indicaciones contextuales en el formulario de pago para usuarios menos familiarizados con el e-commerce.

Capítulo 7

Conclusiones

7.1. Revisión de costes

7.2. Conclusiones

El presente proyecto ha permitido desarrollar una tienda online completa y funcional para la Droguería Mercería Inés, cubriendo desde el catálogo de productos y el proceso de compra con pasarela de pago hasta el panel de administración y el sistema de notificaciones. A continuación se revisa el grado de cumplimiento de cada uno de los objetivos planteados al inicio del trabajo.

El **objetivo principal** —desarrollar una tienda online funcional que permita a los clientes consultar el catálogo, realizar pedidos y efectuar pagos de forma segura— se ha alcanzado en su totalidad. La aplicación implementa el flujo completo de compra, desde el registro del usuario hasta la confirmación del pedido con generación automática de factura en PDF.

Respecto a los **objetivos secundarios**, todos han sido alcanzados satisfactoriamente:

- La plataforma web propia proporciona al negocio una presencia digital independiente de las redes sociales, con un catálogo consultable en cualquier momento y desde cualquier dispositivo.
- La propietaria dispone de un panel de administración desde el que puede gestionar el inventario, actualizar productos con imágenes almacenadas en Cloudinary y hacer seguimiento de los pedidos recibidos.
- El bot de Telegram notifica automáticamente al administrador ante cada nueva compra y emite alertas cuando el stock de un producto desciende por debajo del umbral configurado.
- El desarrollo con **Angular** y **Spring Boot** ha proporcionado una base sólida en las tecnologías empleadas en las prácticas profesionales en **Capgemini**, resultando de aplicación directa en el entorno laboral.
- La solución se ha construido íntegramente con herramientas gratuitas y de código abierto, demostrando que es posible desarrollar un sistema de comercio electrónico profesional sin incurrir en costes de licencias.

Entre los aspectos que han supuesto un mayor reto técnico destaca la integración del **bot de Telegram**. Al tratarse de una tecnología no trabajada previamente, requirió un proceso de aprendizaje específico: primero comprendiendo el protocolo de la API de Telegram y posteriormente implementando el sistema de polling manual con **RestTemplate** para la recepción de comandos y el envío de notificaciones.

A nivel de aprendizaje, este proyecto ha consolidado el conocimiento de la arquitectura cliente-servidor mediante una aplicación real y completa, abarcando tanto el frontend en Angular como el backend en Spring Boot, con toda la integración que ello implica: autenticación JWT, gestión transaccional, pasarela de pago y servicios externos.

Por último, cabe señalar que la propietaria del negocio ha podido ver y probar la aplicación, valorando positivamente tanto el panel de administración como la experiencia de compra. El despliegue en un servidor público queda como trabajo futuro pendiente de acuerdo con sus necesidades y disponibilidad.

7.3. Trabajo futuro

Aunque el sistema desarrollado cubre todos los requisitos funcionales planteados, durante el proceso de desarrollo y las pruebas realizadas se han identificado varias líneas de mejora que podrían abordarse en versiones futuras de la aplicación.

- **Despliegue en producción.** La aplicación está preparada para ser desplegada en un servidor público, pero actualmente solo funciona en entorno local. El siguiente paso natural es publicarla en una plataforma de alojamiento en la nube para que sea accesible a cualquier cliente desde internet.
- **Adaptación para dispositivos móviles.** Aunque la interfaz es funcional en escritorio, una adaptación específica del diseño para teléfonos y tabletas mejoraría significativamente la experiencia de los clientes que acceden desde dispositivos móviles.
- **Integración con servicios de mensajería y envío.** Actualmente el sistema gestiona el pedido y el pago, pero la logística de envío queda fuera del alcance de la aplicación. Integrar un servicio de mensajería permitiría generar etiquetas de envío, comunicar el estado del transporte al cliente y cerrar el ciclo completo de la compra online.
- **Sistema de reseñas de productos.** Permitir que los clientes que hayan adquirido un producto dejen una valoración y un comentario aportaría información útil tanto a otros compradores como a la propietaria para conocer la satisfacción con el catálogo.
- **Procesamiento asíncrono en la creación de pedidos.** Como se observó en las pruebas de rendimiento, el endpoint `POST /api/pedidos/crear` tarda 1.870 ms debido a que el envío del correo de confirmación y la notificación de Telegram se realizan de forma síncrona. Procesarlas de forma asíncrona, por ejemplo mediante `@Async` de Spring, reduciría el tiempo de respuesta percibido por el usuario.
- **Mejora de la usabilidad en el proceso de pago.** Las pruebas de usabilidad evidenciaron que el formulario de *checkout* genera cierta fricción en perfiles con escasa experiencia en compras online. Incorporar indicaciones contextuales y mensajes de ayuda en ese paso haría la experiencia más accesible para ese tipo de usuarios.

Bibliografía

- [1] Wix.com Ltd. Wix Help Center. <https://support.wix.com/>, 2025. Consultado en octubre de 2025.
- [2] Squarespace Inc. Squarespace Help Center. <https://support.squarespace.com/>, 2025. Consultado en octubre de 2025.
- [3] PrestaShop S.A. Documentación Oficial de PrestaShop. <https://devdocs.prestashop-project.org/>, 2025. Consultado en octubre de 2025.
- [4] Google Developers. Angular Documentation. <https://angular.io/docs>, 2025. Consultado en octubre de 2025.
- [5] Meta Platforms, Inc. React Official Documentation. <https://react.dev/>, 2025. Consultado en octubre de 2025.
- [6] Evan You. Vue.js Official Guide. <https://vuejs.org/guide/introduction.html>, 2025. Consultado en octubre de 2025.
- [7] VMware, Inc. Spring Boot Documentation. <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>, 2025. Consultado en octubre de 2025.
- [8] OpenJS Foundation. Node.js Documentation. <https://nodejs.org/en/docs/>, 2025. Consultado en octubre de 2025.
- [9] Django Software Foundation. Django Official Documentation. <https://docs.djangoproject.com/en/5.0/>, 2025. Consultado en octubre de 2025.
- [10] python-telegram-bot Contributors. python-telegram-bot Documentation. <https://docs.python-telegram-bot.org/>, 2025. Consultado en octubre de 2025.
- [11] Telegraf Contributors. Telegraf Documentation. <https://telegraf.js.org/>, 2025. Consultado en octubre de 2025.
- [12] Ruben Bermudez. TelegramBots Java Library. <https://github.com/rubenlagus/TelegramBots>, 2025. Consultado en octubre de 2025.
- [13] Oracle Corporation. MySQL Documentation. <https://dev.mysql.com/doc/>, 2025. Consultado en octubre de 2025.
- [14] MongoDB Inc. MongoDB Manual. <https://www.mongodb.com/docs/>, 2025. Consultado en octubre de 2025.
- [15] PayPal Holdings Inc. PayPal Developer Documentation. <https://developer.paypal.com/docs/>, 2025. Consultado en octubre de 2025.

-
- [16] Redsys Servicios de Procesamiento S.L. Manual de Integración de Pasarela de Pagos. <https://pagosonline.redsys.es>, 2025. Consultado en octubre de 2025.
 - [17] Stripe Inc. Stripe API Documentation. <https://stripe.com/docs/api>, 2025. Consultado en octubre de 2025.
 - [18] Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*. PMI, Newtown Square, Pennsylvania, 2021.